



南京工业大学
NANJING TECH
UNIVERSITY

用户数据采集与关联分析

(结课作业)

信管2301 吴安杰

202321054023





南京工业大学
NANJING TECH
UNIVERSITY

01 文本数据分析



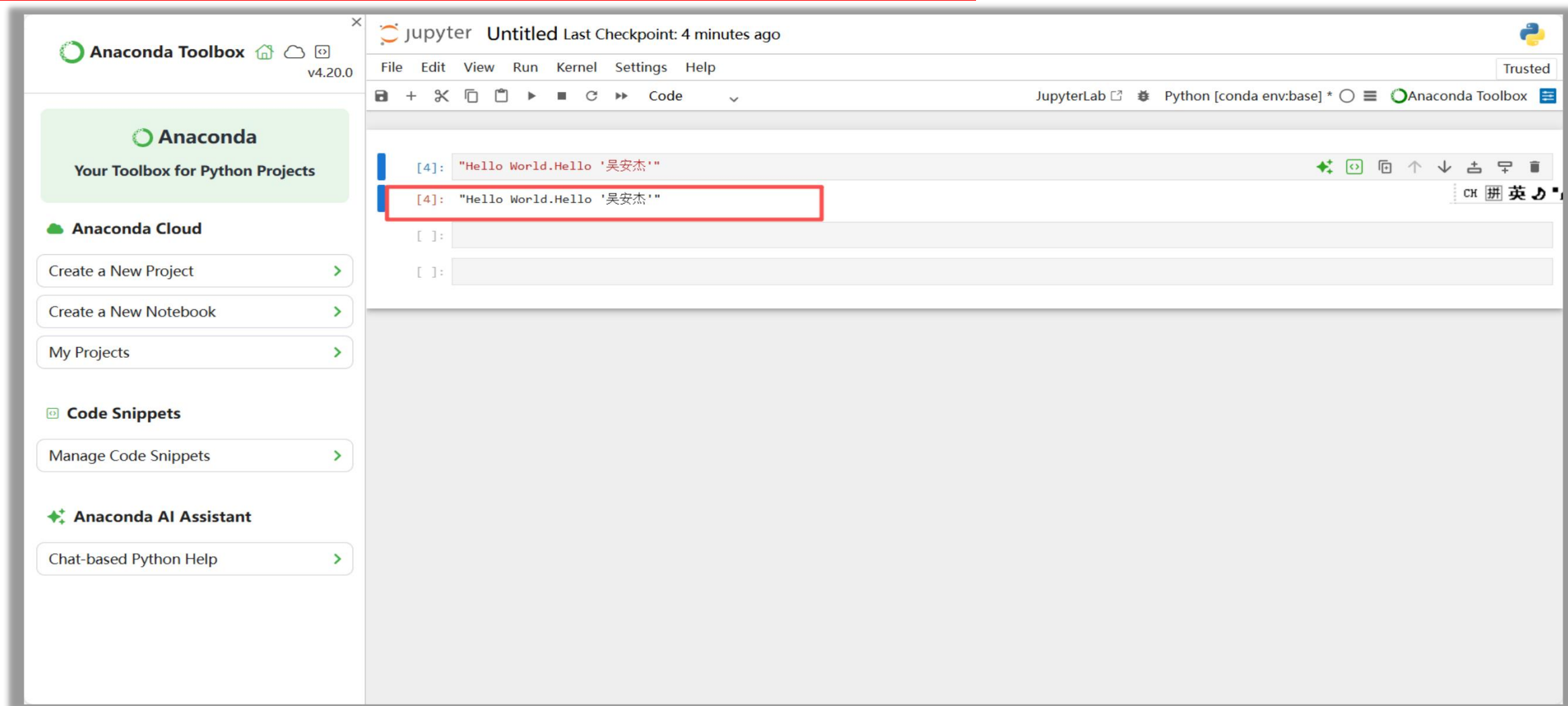
第一讲 课程导言与分词

任务一：清华大学分词演示系统



第一讲 课程导言与分词

任务二：anaconda 输出语句



第一讲 课程导言与分词

任务三：课后作业的四分代码运行（001-002）



JupyterLab 001-word_cut_基本分词 Last Checkpoint: 3 months ago

File Edit View Run Kernel Settings Help Trusted

Python [conda env:base] * Anaconda Toolbox

```
Loading model from cache C:\Users\10293\AppData\Local\Temp\jieba.cache
Loading model cost 0.791 seconds.
Prefix dict has been built successfully.
南京 大学生 都 爱 南京市 长江大桥
```

[5]: `seg_list = jieba.cut("南京大学生都爱南京市长江大桥")`

[6]: `print(' '.join(seg_list))`

南京*大学生*都*爱*南京市*长江大桥

2 加入用户词典

[7]: `seg_list1 = jieba.cut("吴志祥是南京工业大学青年教师, 他对那种二次元小魔仙是无感的, 这怎么行?")`

[8]: `print(' '.join(seg_list1))`

吴志祥#是#南京#工业#大学#青年教师#, #他#对#那种#二次元#小#魔仙#是#无感#的#, #这#怎么#行#?

载入词典

[9]: `jieba.load_userdict('dict.txt')`

[10]: `seg_list = jieba.cut("吴志祥是南京工业最好大学青年教师, 经济与管理学院的, 他对那种二次元小魔仙是无感的, 这怎么行? python看上去还有点人性")`

[11]: `print(' '.join(seg_list))`

吴志祥是#南京工业最好大学#青年教师#, #经济#与#管理#学院#的#, #他#对#那种#二次元#小#魔仙#是#无感#的#, #这#怎么#行#? #python#看上去#还有点#人性

3 载入停用词表

[12]: `seg_list = jieba.cut("没有使用停用词表的分词结果, 就会有很多没有用的词啊, 虚词、感叹词什么的")`



JupyterLab 002-word_cut_科学家文本 Last Checkpoint: 7 months ago

File Edit View Run Kernel Settings Help Not Trusted

Python [conda env:base] * Anaconda Toolbox

Markdown

```
[5]: import jieba
```

[6]: `seg_list_huang = jieba.cut('黄旭华, 1926年3月12日出生于广东省汕尾市, 原籍广东省揭阳市。1949年毕业于上海交通大学。历任北京海军核潜艇研究室副总工程师、中船重工集团公司核潜艇总体/研究/设计所/研究员、/名誉/所长、/1994年/当选/为中国工程院院士。')`

[7]: `print(' '.join(seg_list_huang))`

黄旭华/, /1926年/3月/12日/出/生/于/广东省/汕尾市/, /原籍/广东省/揭阳市/, /1949年/毕业/于/上海交通大学/, /历任/北京/海军/核潜艇/研究室/副总工程师/, /中/船/重工/集团/公司/核潜艇/总体/研究/设计所/研究员/, /名誉/所长/, /1994年/当选/为/中国工程院院士/。

[5]: # 加入用户词典

[6]: `jieba.load_userdict('dict.txt')`

[7]: `seg_list_huang = jieba.cut('黄旭华, 1926年3月12日出生于广东省汕尾市, 原籍广东省揭阳市。1949年毕业于上海交通大学。历任北京海军核潜艇研究室副总工程师、中船重工集团公司核潜艇总体/研究/设计所/研究员、/名誉/所长、/1994年/当选/为中国工程院院士。')`

[8]: `print(' '.join(seg_list_huang))`

黄旭华/, /1926年/3月/12日/出/生/于/广东省/汕尾市/, /原籍/广东省/揭阳市/, /1949年/毕业/于/上海交通大学/, /历任/北京/海军/核潜艇/研究室/副总工程师/, /中船重工集团公/司/核潜艇/总体/研究/设计所/研究员/, /名誉/所长/, /1994年/当选/为/中国工程院院士/。

[9]: # 加入词典之后, 哪些词汇就分出来了呢?

[10]: # 使用停用词表

[11]: `stopwords = [line.strip() for line in open('stop_words.txt', 'r', encoding='utf-8').readlines()]`

[12]: `stopwords = open('stop_words.txt', 'r', encoding='utf-8').read()`
`stopwords = stopwords.split('\n')`

[13]: `stopwords`

[13]: ['的', '了', '是', '啊', ',', '.', '!', '停用']

第一讲 课程导言与分词

任务三：课后作业的四分代码运行（003-004）

```
Jupyter 003-NER-企业年报-数字技术-安全管理 Last Checkpoint: 7 months ago
File Edit View Run Kernel Settings Help
JupyterLab Python [conda env:base] Anaconda Toolbox

# 定义要统计的关键词
target_words = ['数字化', '智能化', '安全']

# 统计词频
word_counts = Counter(words)

# 输出特定词汇的词频统计结果
print("特定词汇词频统计结果:")
for word in target_words:
    print(f"{word}: {word_counts[word]}次")

特定词汇词频统计结果:
'数字化': 2次
'智能化': 3次
'安全': 2次

# 输出所有词汇的词频 (按频率排序)
print("\n所有词汇词频统计 (前20个):")
for word, count in word_counts.most_common(20):
```

```
Jupyter 004_使用大语言模型提取科技文献中的实体 Last Checkpoint: 7 months ago
File Edit View Run Kernel Settings Help
JupyterLab Python [conda env:base] Anaconda Toolbox

if response.status_code == 200:
    result = response.json()
    try:
        entities = result['choices'][0]['message']['content']
        print("提取到的实体和专业术语:")
        print(entities)
    except KeyError:
        print("无法解析API响应, 原始响应:")
        print(result)
else:
    print(f"请求失败, 状态码: {response.status_code}")
    print(response.text)

提取到的实体和专业术语:
'''json
{
  "理论": [
    "肿瘤免疫微环境",
    "T细胞耗竭",
    "免疫编辑理论"
  ],
  "方法": [
    "单细胞RNA测序",
    "细胞亚群聚类",
    "轨迹分析",
    "pseudotime推断",
    "细胞间通讯网络构建"
  ],
  "工具": [
    "Seurat",
    "Monocle3",
    "CellChat"
  ],
  "专业术语": [
    "TIME",
    "scRNA-seq",
    "非小细胞肺癌",
    "免疫抑制信号通路",
    "PD-1/PD-L1",
    "TGF-β途径",
    "个性化免疫治疗"
  ]
}'''
```

第一讲 课程导言与分词

任务四：实体抽取论文《基于学术论文全文的研究方法实体自动识别研究》

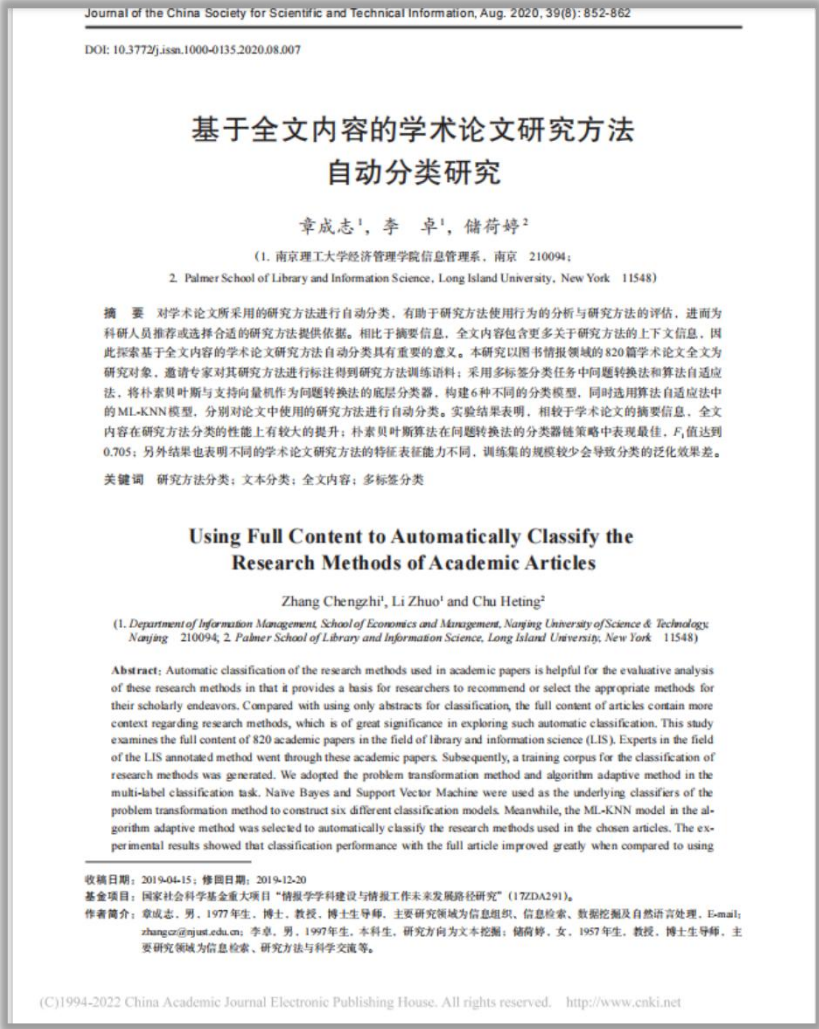
阅读总结：本文《基于学术论文全文的研究方法实体自动识别研究》针对学术文本中研究方法实体的自动抽取问题展开深入探讨。研究将研究方法实体区分为“论文使用的研究方法”与“论文引用的研究方法”，并系统比较了八种基于深度学习的序列标注模型。

实验结果表明，采用字符向量与Word2Vec领域词向量相结合，并融合条件随机场的双向长短时记忆网络模型，在识别任务中取得了最优性能，尤其在论文使用方法识别上F1值达到65.42%。

研究进一步以《情报学报》近十年（2009–2018年）的1170篇论文为实证数据，进行了大规模的实体识别与统计分析。结果发现，在情报学领域，实验法相关的研究方法在使用频次与引用频次上均显著高于其他类别，反映出该学科在方法上倾向于实证与计算导向。

文章亦指出当前研究面临的挑战，主要包括标注数据规模有限、实体识别准确率仍有提升空间。未来工作将考虑引入半监督学习与规则辅助方法，以降低对标注数据的依赖，并进一步扩大语料规模，深化对学科方法演进模式的分析。

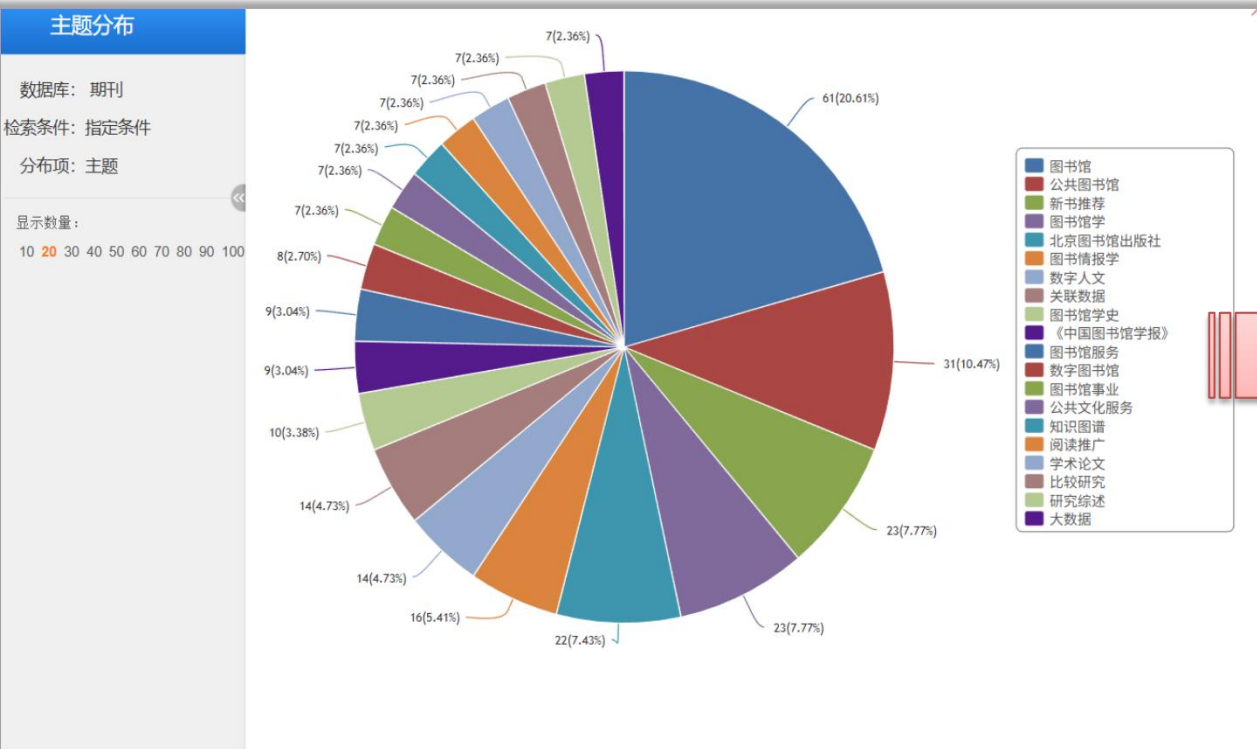
该研究不仅为学术文本中的方法实体识别提供了有效的技术路径，也为学科方法论的量化研究提供了新的分析工具与数据基础。



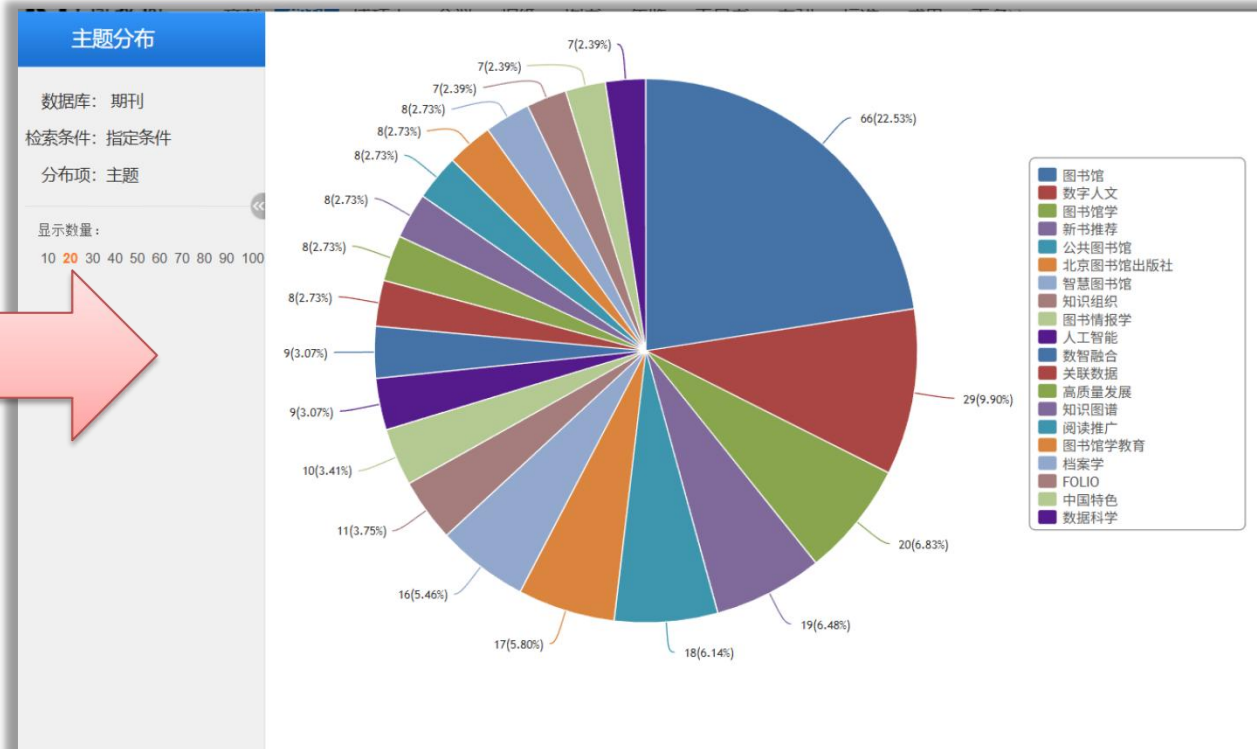
第二讲 词频统计

任务一：关于期刊《中国图书馆学报》的主题趋势变化

2014-2019

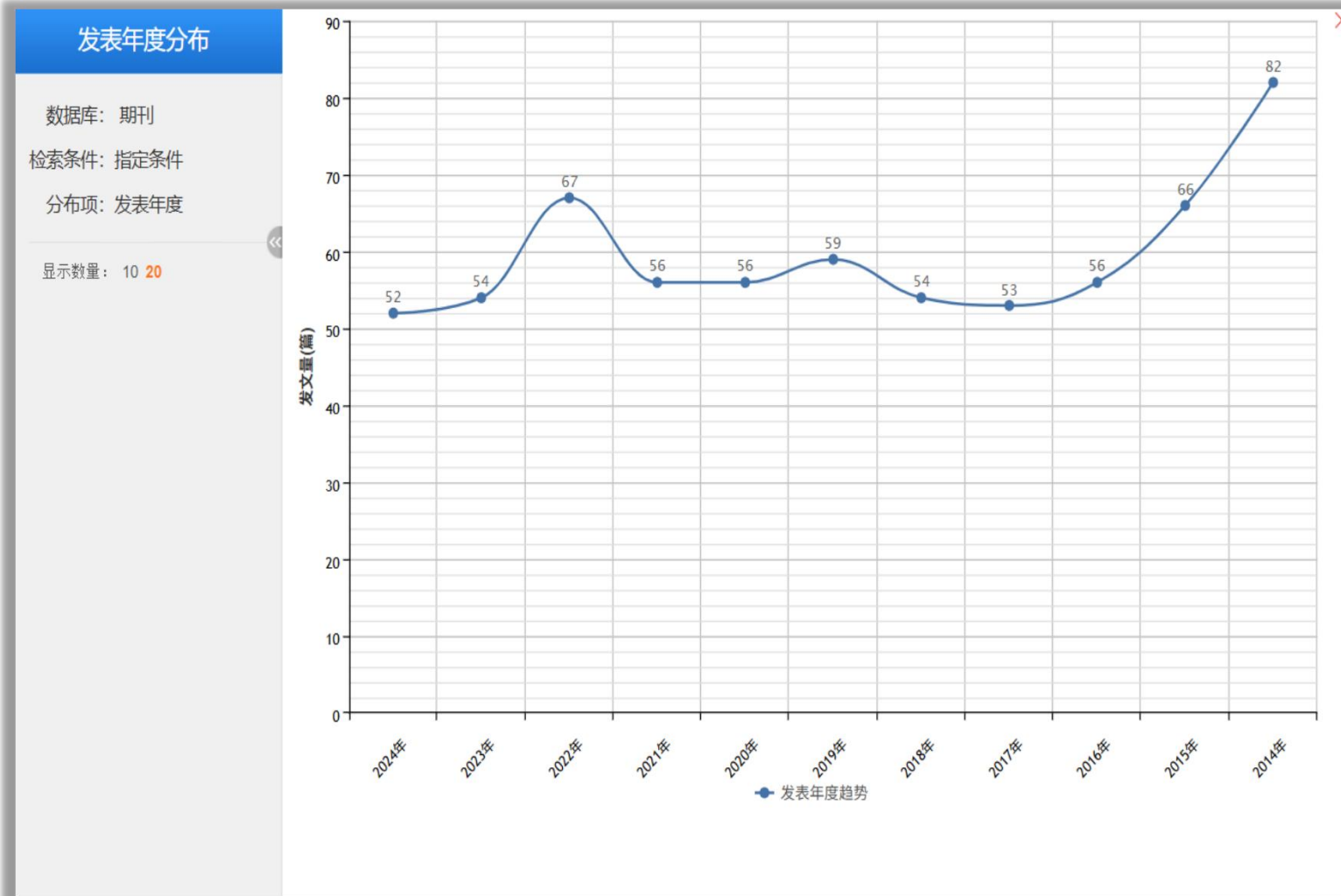


2019-2024



第二讲 词频统计

任务一：关于期刊《中国图书馆学报》的主题趋势变化



前五年主题集中于数字图书馆、信息组织、信息资源建设等基础性数字化转型。

后五年集中于大数据、人工智能、数据挖掘、知识图谱等技术驱动的研究成为绝对主流，研究范式转向计算与智能。

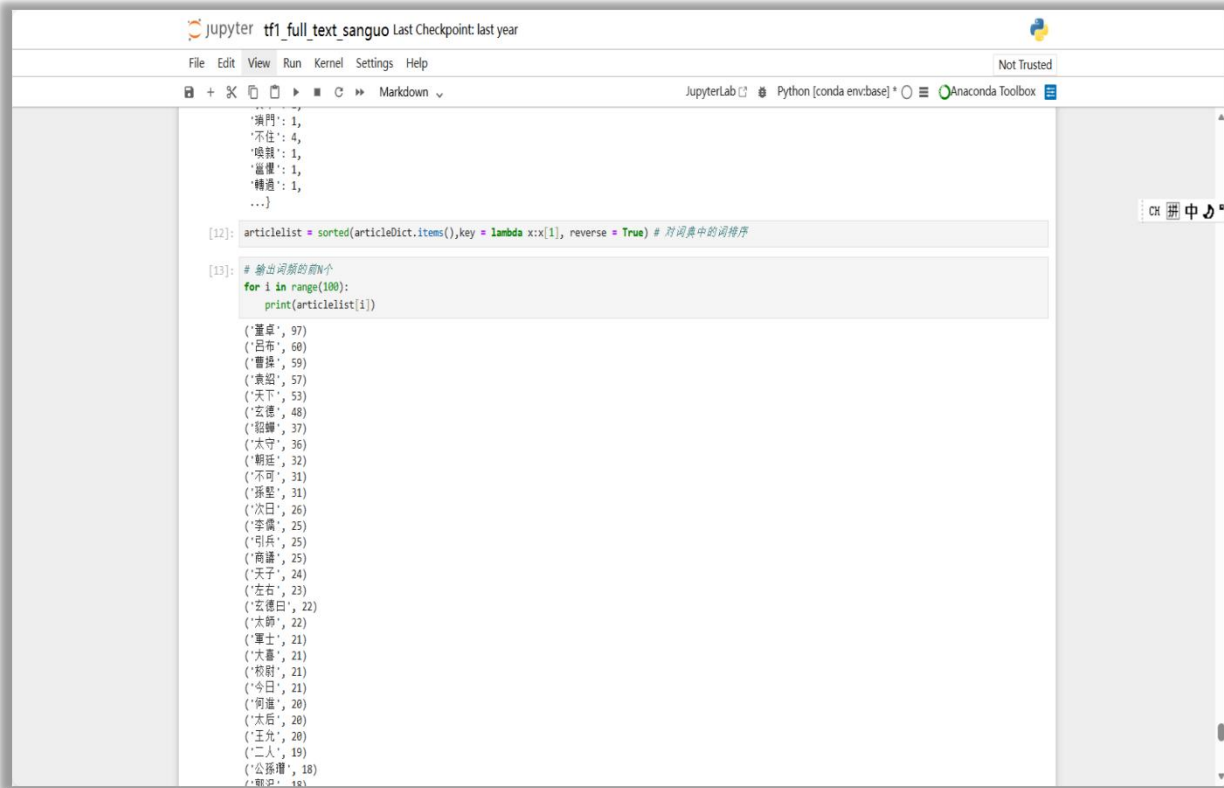
主题呈现深度融合与价值拓展：一方面，智慧图书馆、元宇宙、生成式AI等前沿探索持续；另一方面，公共文化服务、高质量发展、可持续发展、数据治理等宏观战略与社会价值议题显著增加，体现了学科从技术赋能向社会责任与理论构建的回归。

整体上，该刊始终引领国内图情领域前沿，完美映射了从“信息资源管理”到“数据与智能驱动”，再迈向“深度融合与价值创新”的学科发展主线。

第二讲 词频统计

任务二：全文词频统计与指定类型词频统计

全文词频统计

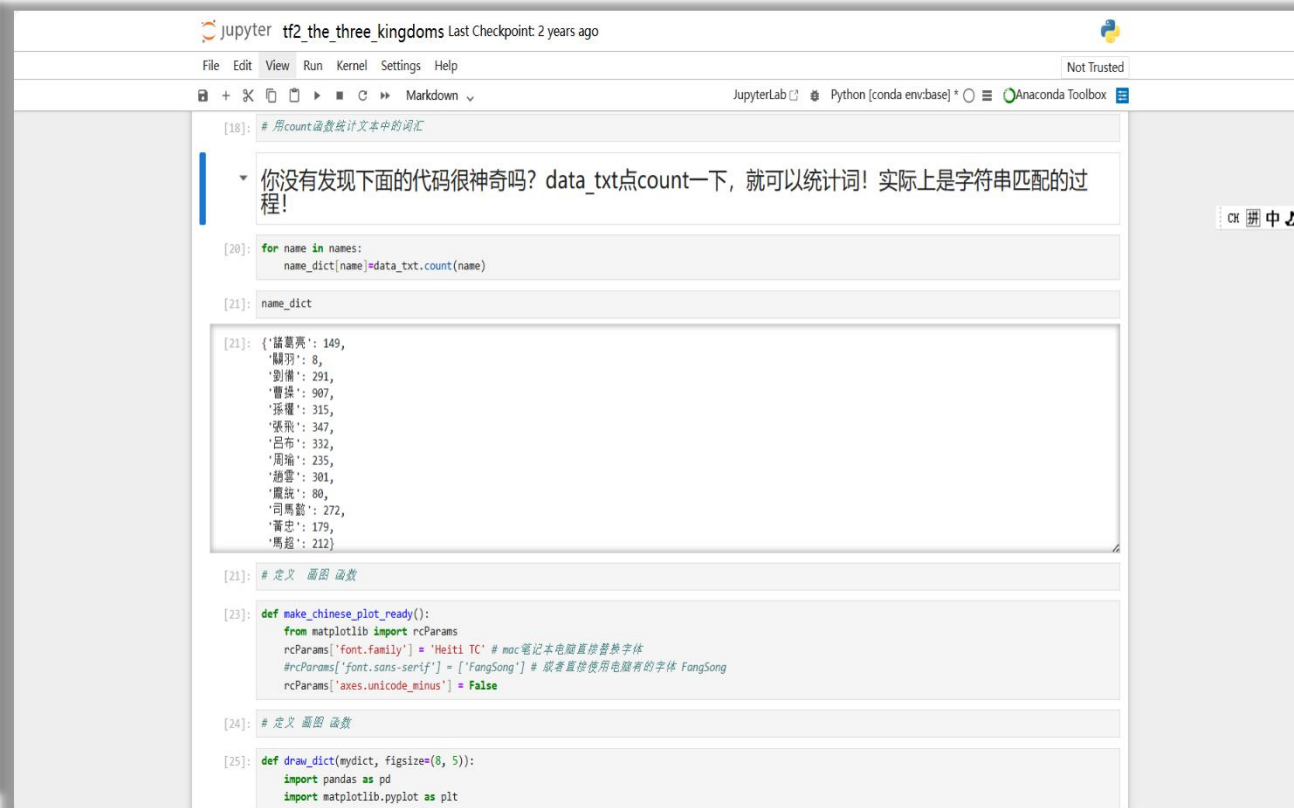


```
jupyter tf1_full_text_sanguo Last Checkpoint: last year
File Edit View Run Kernel Settings Help
JupyterLab Python [conda env:base] Anaconda Toolbox

[12]: articlelist = sorted(articleDict.items(), key = lambda x:x[1], reverse = True) # 对词典中的词排序
[13]: # 输出词频的前N个
for i in range(100):
    print(articlelist[i])

('董卓', 97)
('吕布', 60)
('曹操', 59)
('袁绍', 57)
('天下', 53)
('玄德', 48)
('貂蝉', 37)
('太守', 36)
('朝廷', 32)
('不可', 31)
('派兵', 31)
('次日', 26)
('李儒', 25)
('引兵', 25)
('曹操', 25)
('天子', 24)
('左右', 23)
('玄德曰', 22)
('太师', 22)
('董卓', 21)
('大喜', 21)
('校尉', 21)
('今日', 21)
('何人', 20)
('太后', 20)
('王允', 20)
('二人', 19)
('公孫權', 18)
('觀之', 18)
```

指定类型词频统计



```
jupyter tf2_the_three_kingdoms Last Checkpoint: 2 years ago
File Edit View Run Kernel Settings Help
JupyterLab Python [conda env:base] Anaconda Toolbox

[18]: # 用count函数统计文本中的词汇
你没有发现下面的代码很神奇吗？data_txt点count一下，就可以统计词！实际上是字符串匹配的过程！

[20]: for name in names:
        name_dict[name]=data_txt.count(name)

[21]: name_dict

[21]: {'诸葛亮': 149,
      '关羽': 8,
      '刘备': 291,
      '曹操': 907,
      '孙权': 315,
      '张辽': 347,
      '吕布': 332,
      '周瑜': 235,
      '赵云': 301,
      '曹操': 80,
      '司马懿': 272,
      '董忠': 179,
      '马超': 212}

[21]: # 定义 画图 函数

[23]: def make_chinese_plot_ready():
        from matplotlib import rcParams
        rcParams['font.family'] = 'Heiti TC' # mac笔记本电脑直接替换字体
        rcParams['font.sans-serif'] = ['FangSong'] # 或者直接使用电脑有的字体 FangSong
        rcParams['axes.unicode_minus'] = False

[24]: # 定义 画图 函数

[25]: def draw_dict(mydict, figsize=(8, 5)):
        import pandas as pd
        import matplotlib.pyplot as plt
```

第二讲 词频统计

任务三：功勋科学家黄旭华文本的全文词频统计与指定类型词频统计

全文词频统计

```
Jupyter tf1_full_text_huangxuhua Last Checkpoint: 2 years ago

File Edit View Run Kernel Settings Help

Python [conda env:base] * Anaconda Toolbox

[12]: # 因为采用的是循环的方法，把所有的词都循环一遍，长度大于1，所以速度很慢！

[14]: for w in articleSet:
        if len(w)>1:
            articleDict[w] = words.count(w) # 字典的key-value配对

[15]: articleDict

[15]: {'揭示': 1,
      '加入': 2,
      '承担': 2,
      '主旨': 1,
      '当时': 1,
      '性能': 2,
      '整编': 1,
      '仿制': 1,
      '齐心协力': 1,
      '组织': 2,
      '命名': 1,
      '七两': 1,
      '包括': 3,
      '清晰': 1,
      '动荡': 1,
      '还有': 1,
      '树基': 1,
      '品德': 2,
      '成立': 1,
      '实物': 5,
      '国家': 3,
      '工程': 11,
      '直至': 1,
      '尽可能': 1,
      '1989': 1,
      '一部分': 1,
      '人员': 3,
      '小组': 14,
      '本章': 1}
```

指定类型词频统计

你没有发现下面的代码很神奇吗？data_txt点count一下，就可以统计词！实际上是字符串匹配的过程！

```
[11]: for term in terms:
        terms_dict[term]=data_txt.count(term)

[12]: terms_dict

[12]: {'黄旭华': 59, '核潜艇': 32, '国立交通大学': 3}

[23]: # 定义 画图 函数

[33]: def make_chinese_plot_ready():
        from matplotlib import rcParams
        rcParams['font.family'] = 'Heiti TC' # mac 笔记本电脑直接替换字体
        rcParams['font.sans-serif'] = ['FangSong'] # 或者直接使用电脑有的字体 FangSong
        rcParams['axes.unicode_minus'] = False

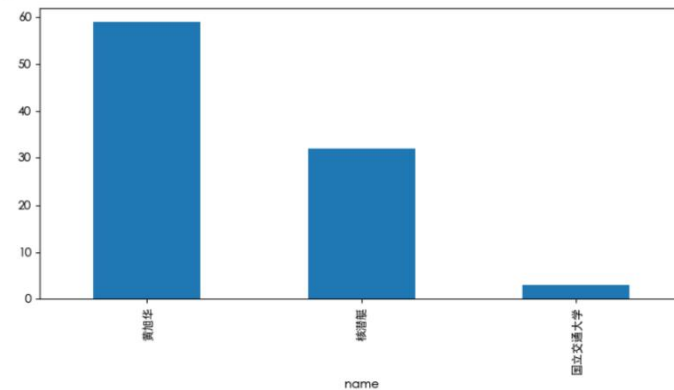
[34]: # 定义 画图 函数

[35]: def draw_dict(mydict, figsize=(8, 5)):
        import pandas as pd
        import matplotlib.pyplot as plt
        make_chinese_plot_ready()
        df = pd.DataFrame(list(mydict.items()), columns=['name', 'times'])
        df.set_index('name')['times'].sort_values(ascending=False).plot(kind='bar', figsize=figsize) # 做好排序
        plt.tight_layout()

[16]: # %pylab inline

[17]: %matplotlib inline

[18]: draw_dict(terms_dict)
```



作业

- 请完成以上两个python的代码
- 请把以上文本替换成文件夹中的《科学家博物馆-黄旭华传记序言.txt》，完成该文档中的全文词频统计，以及指定词汇统计，如统计“黄旭

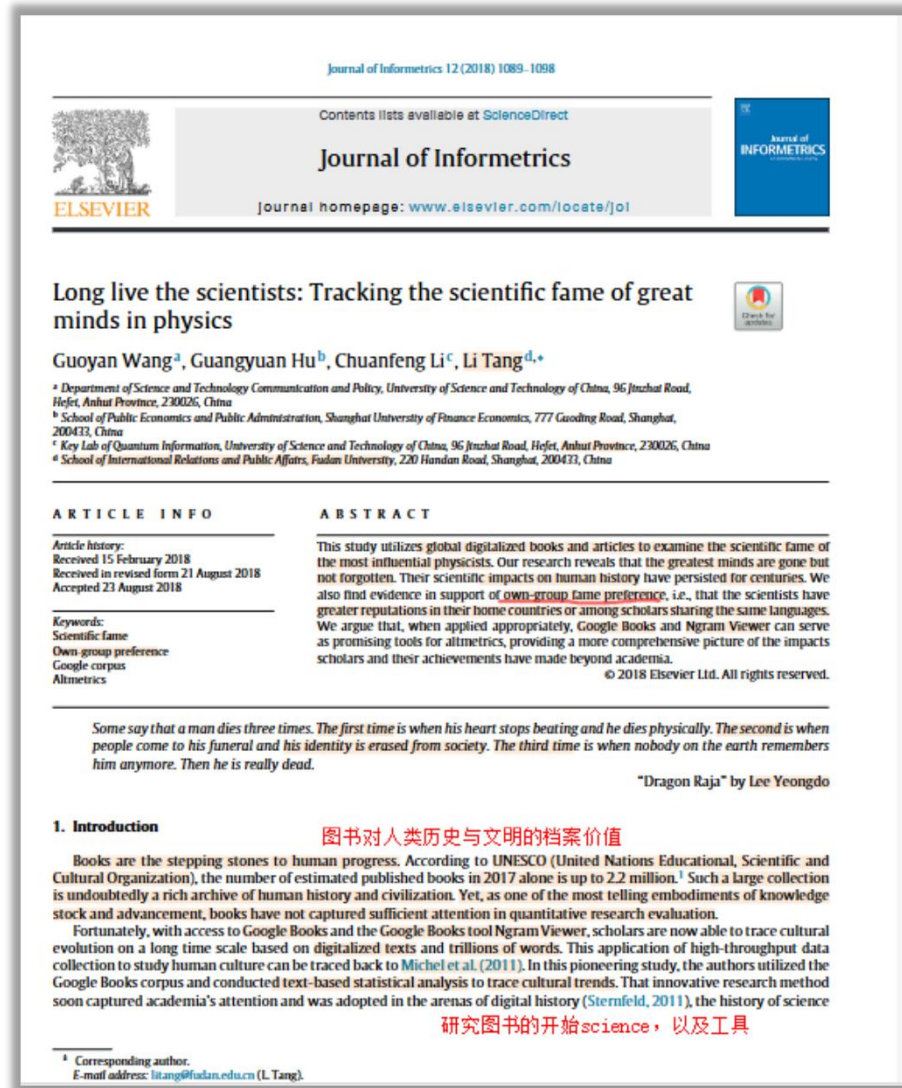
第二讲 词频统计

任务四：2018-Wang 等 - Long live the scientists Tracking the scientific reading 总结.

阅读总结：

本研究利用谷歌图书与谷歌学术的大数据，创新性地量化了牛顿、爱因斯坦等物理学巨匠的长期科学声誉。核心发现如下：第一，伟大科学家的声誉具有超长生命力，在其逝世后仍能持续数个世纪。第二，声誉存在显著的“同群偏好”，即科学家在本土或同语言文化群体中享有更高声誉（如在英国书籍中牛顿声誉更高），这解释了基于局部调查与全球数据结论的差异。第三，通过共现分析发现，科学家的公众声誉与其核心科学成就紧密关联（如爱因斯坦28%的提及与相对论相关），但也有相当部分源于其他贡献与生平轶事。

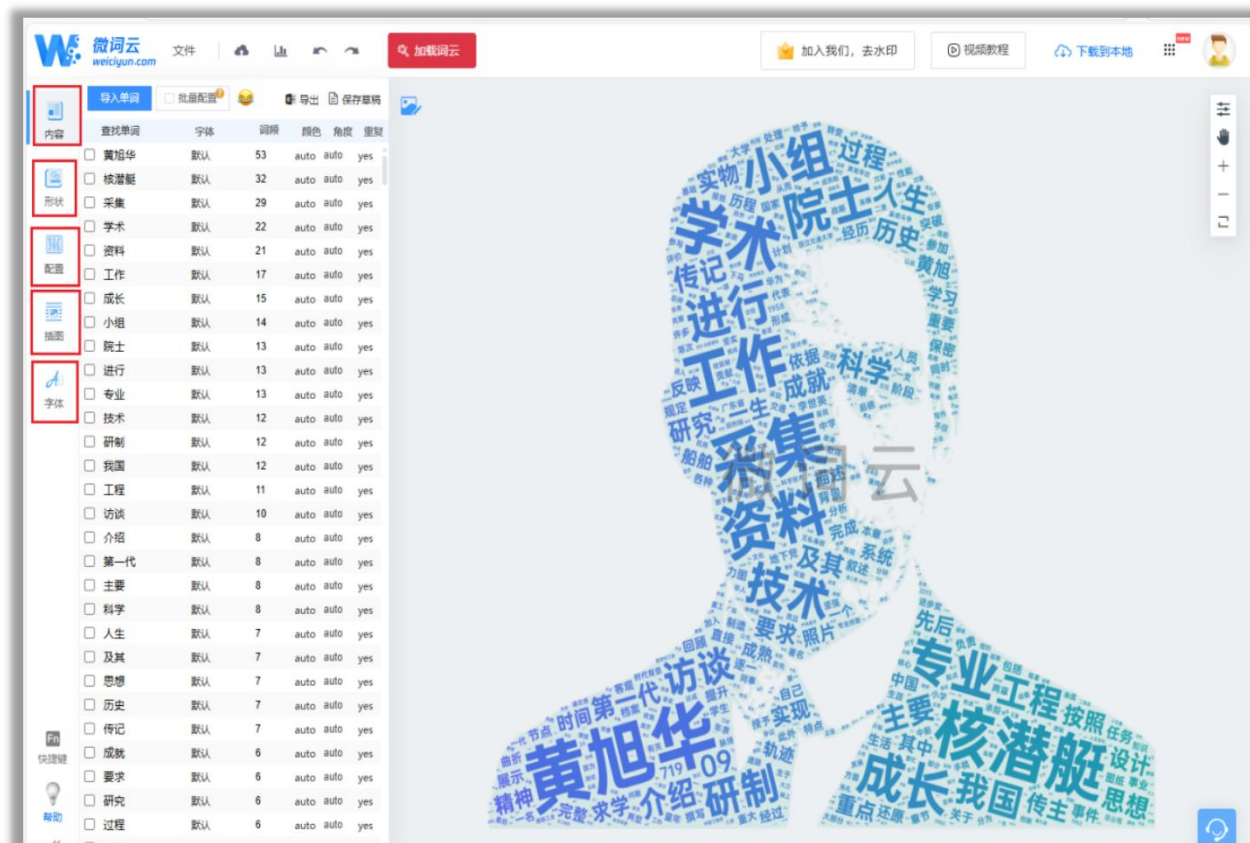
该研究的核心贡献在于方法论创新，它论证了数字化书籍可作为衡量科学家长期社会影响力的替代计量学工具，为超越传统引文分析、评估科学文化遗产提供了新路径。同时，研究也指出了数据存在语言偏差、名字歧义等局限性，使其更适用于评估已有广泛声誉的杰出人物。



第三讲 词云与可视化

任务一：绘制词云图

微词云软件：

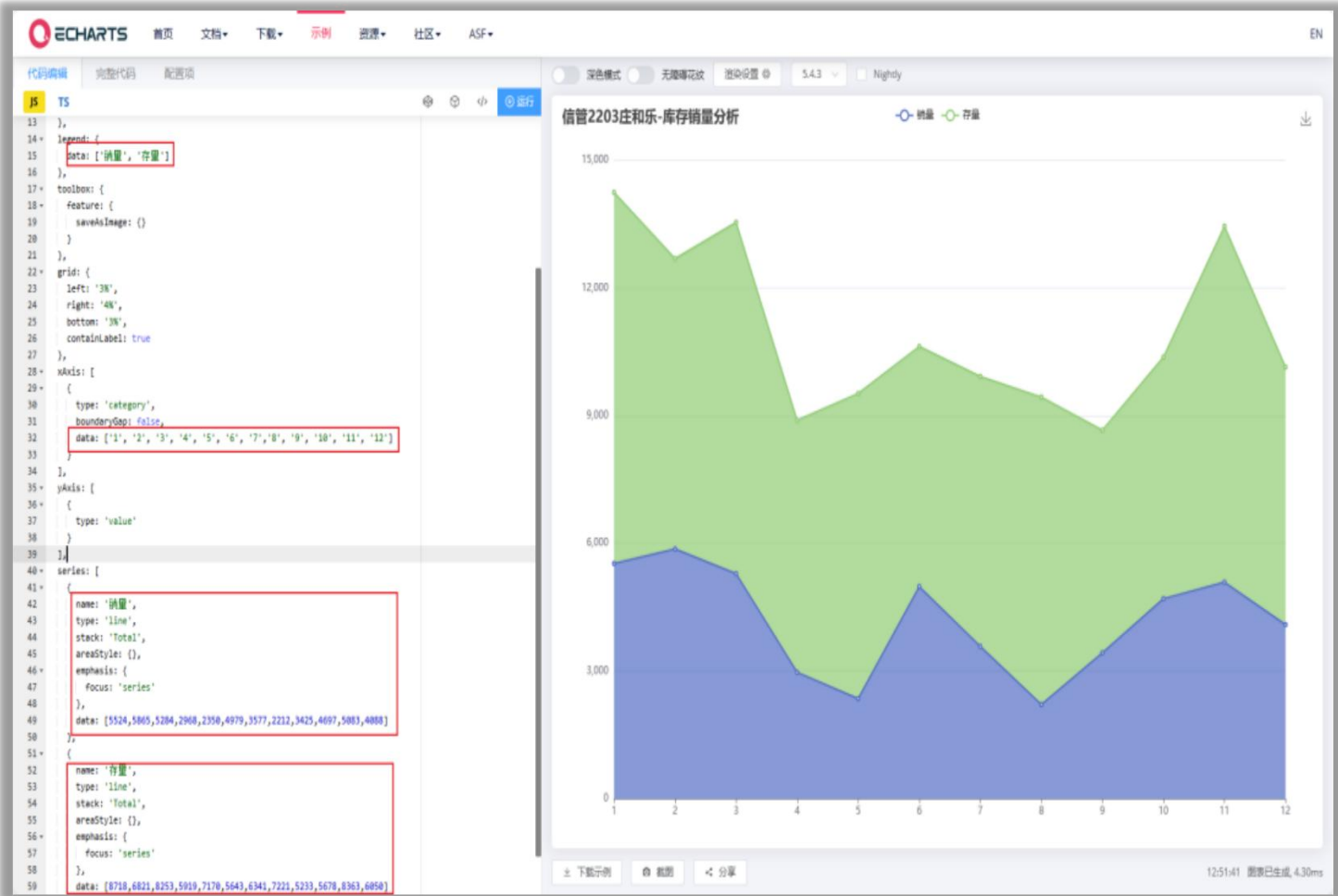


上图为导出的词云图结果，通过词云图可以发现黄旭华的字体最大，说明在文本中出现的次数最多，因该文本为黄旭华先生的传记，因此符合基本情况，核潜艇的大小其次，说明了核潜艇作为黄旭华先生的研究内容，突出了传记的侧重点，其余大部分词汇均与学术研究相关，由此可见黄旭华先生一生投入学术科研事业。

第三讲 词云与可视化

任务二：使用ECharts制作3个图

堆叠面积图



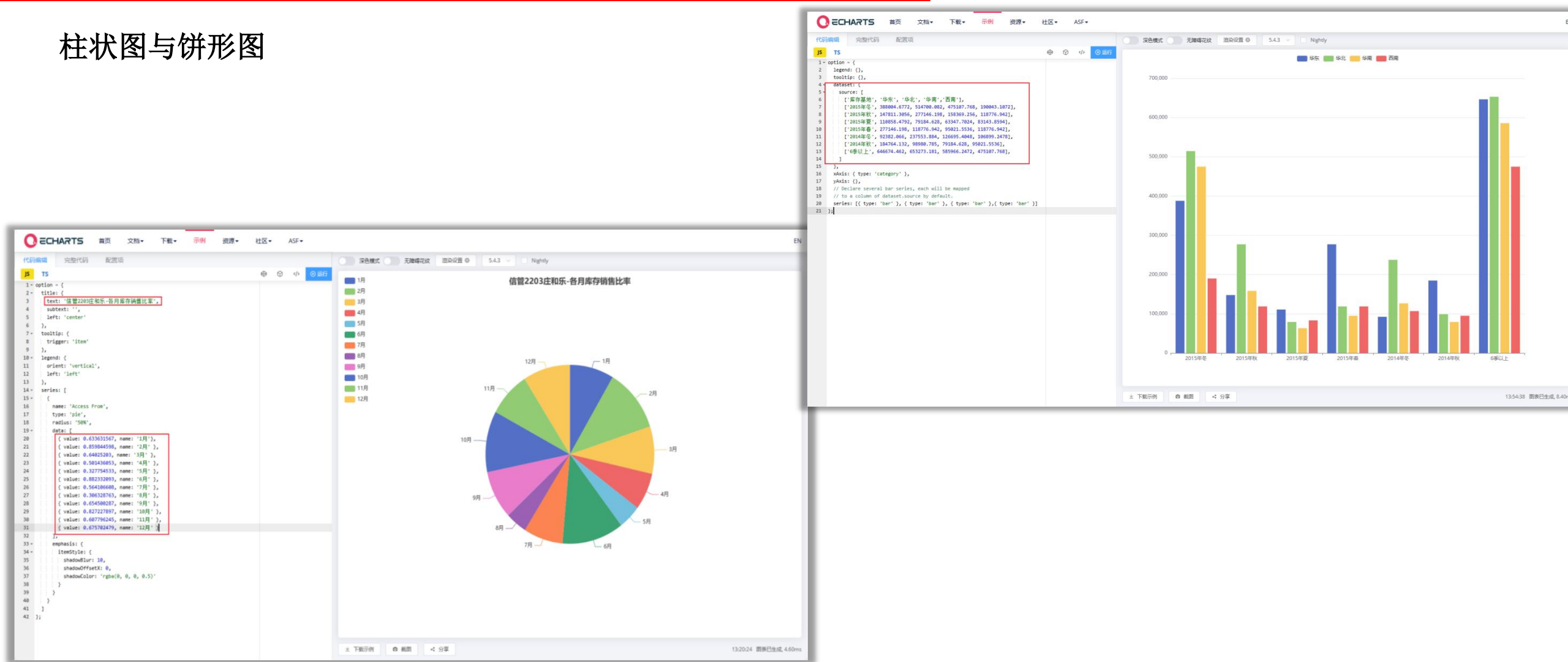
数据来源：

	A	B	C	D	E	F
1	月份	销量	存量	库存金额	库销比	
2	12	4088	6050	6598719	0.675702	
3	11	5083	8363	3383463	0.607796	
4	10	4697	5678	6702705	0.827228	
5	9	3425	5233	7219684	0.6545	
6	8	2212	7221	3550677	0.306329	
7	7	3577	6341	8341596	0.564107	
8	6	4979	5643	9285074	0.882332	
9	5	2350	7170	6574930	0.327755	
10	4	2968	5919	6244773	0.501436	
11	3	5284	8253	7603899	0.640252	
12	2	5865	6821	5834797	0.859845	
13	1	5524	8718	5372643	0.633632	
14						
15						

第三讲 词云与可视化

任务二：使用ECharts制作3个图

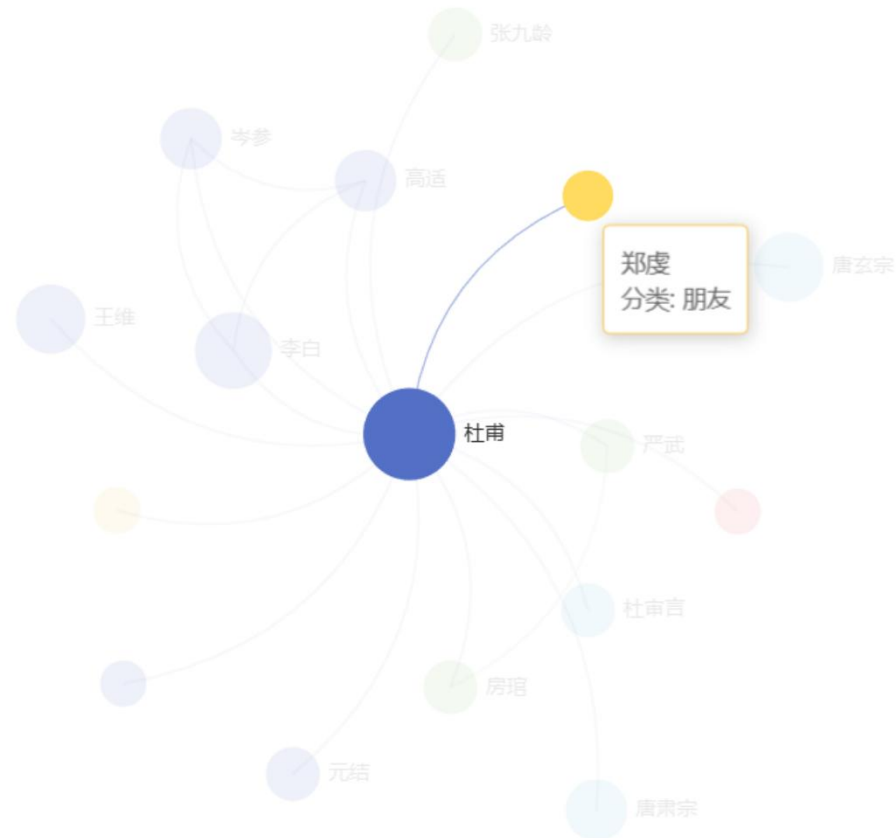
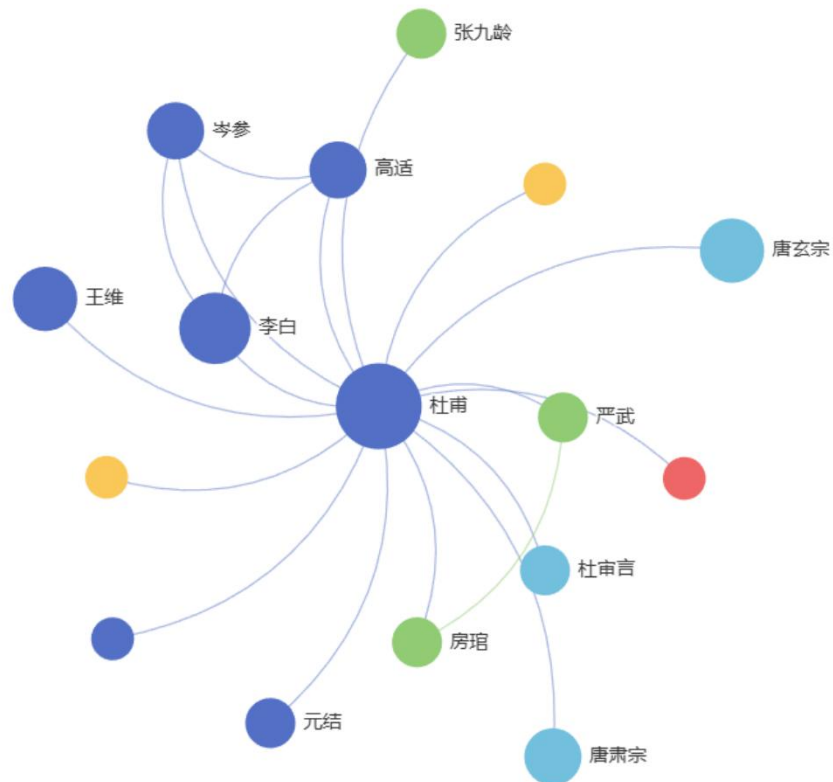
柱状图与饼形图



第三讲 词云与可视化

任务二：使用ECharts制作3个图

杜甫人物关系图

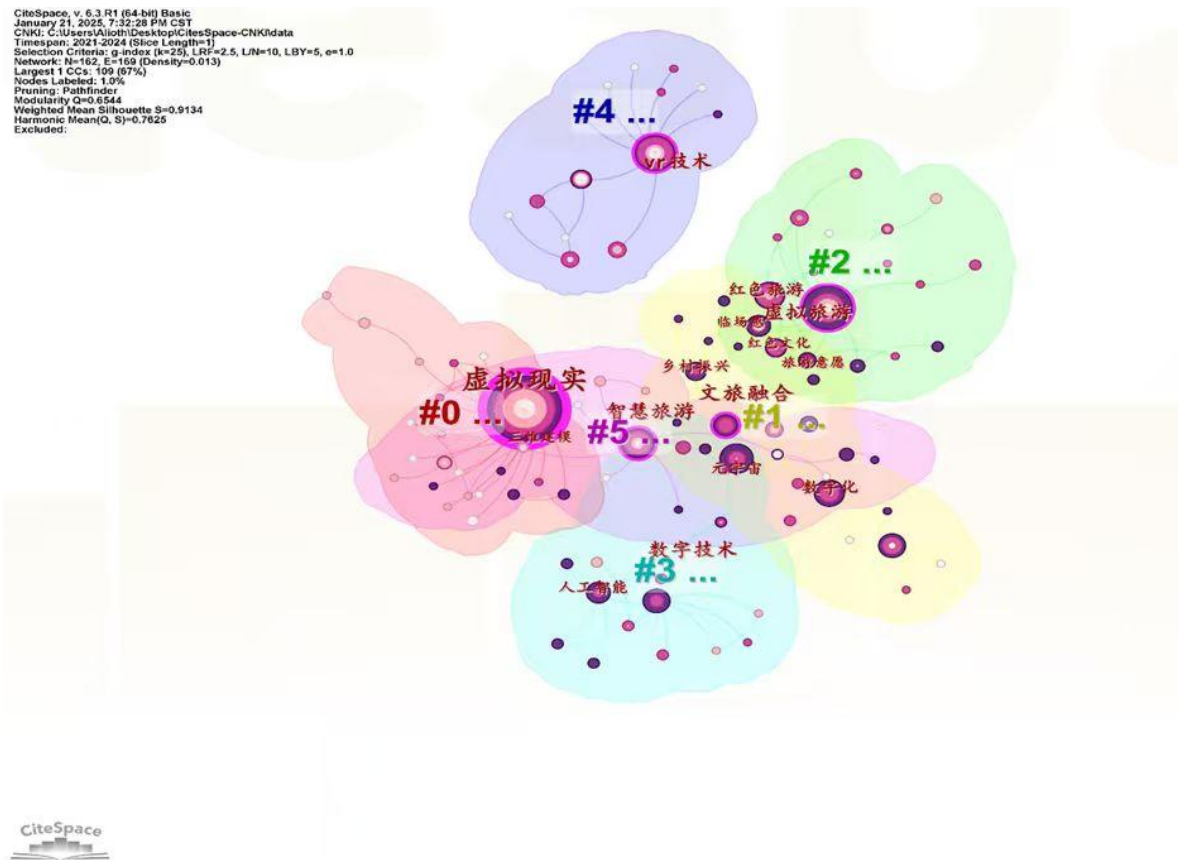
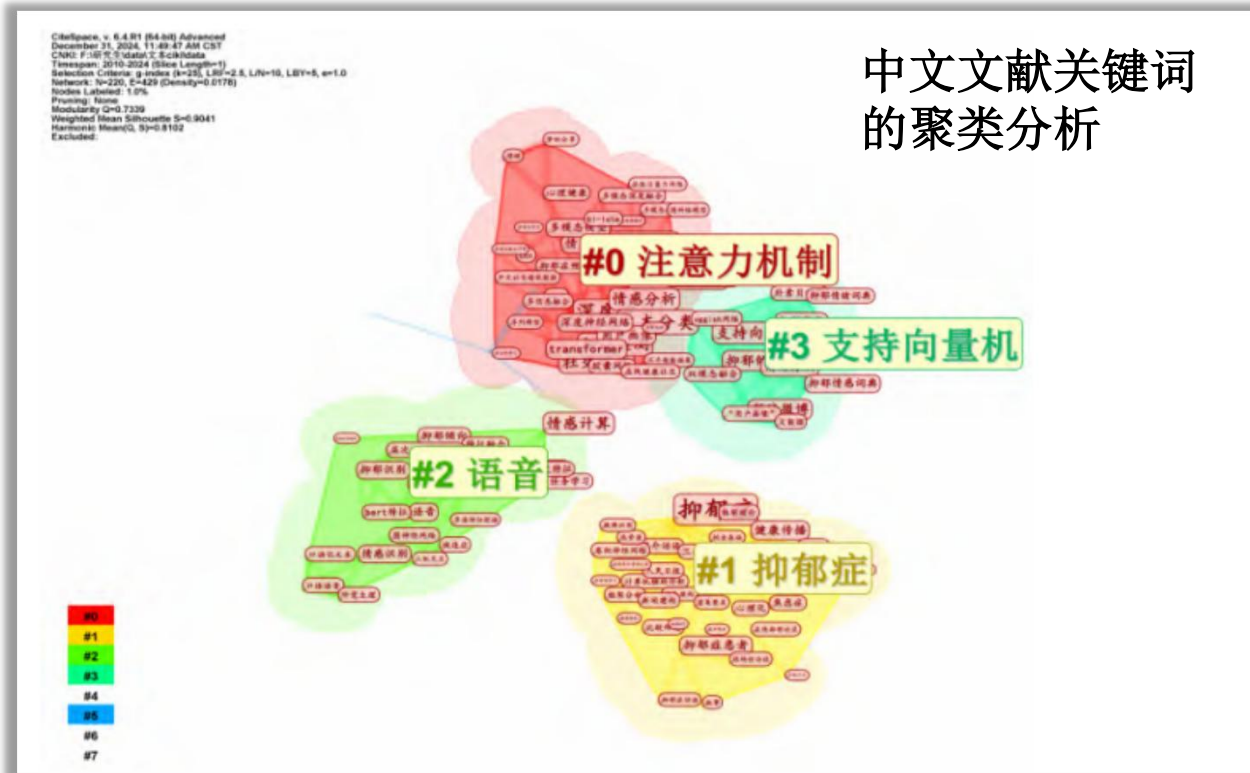


第三讲 词云与可视化

任务三：使用Citespace制作感兴趣的图谱

对于文本识别抑郁领域文献的分析-基于citespace

VR旅游使用citespace做的分析



第三讲 词云与可视化

任务四：运行词云图代码

```
[12]: wordcloud = WordCloud().generate(text_cn_word) # 关键代码2
```

```
[13]: %pylab inline
```

PyLab is deprecated, use %matplotlib inline and import the required libraries.
Populating the interactive namespace from numpy and matplotlib

```
[14]: # 原路一起把以上的代码再重新输入一遍吧，再把字体加入
from wordcloud import WordCloud
wordcloud_cn = WordCloud(font_path="simsun.ttf").generate(text_cn_word)
%pylab inline
import matplotlib.pyplot as plt
plt.imshow(wordcloud_cn, interpolation='bilinear')
plt.axis('off')
```

PyLab is deprecated, use %matplotlib inline and import the required libraries.
Populating the interactive namespace from numpy and matplotlib

```
[14]: (-0.5, 399.5, 199.5, -0.5)
```



做第一个改进

词云图里面有很多我们不想要的词！上停用词表吧

```
[15]: stopwords = open('stop_words.txt', 'r', encoding='utf-8').read()
stopwords = stopwords.split('\n')
```

```
[16]: f_cn = open('功勋科学家黄旭华传记序言.txt', encoding='utf-8') # mac电脑的编码是GB18030, windows可以改成utf-8
```

核潜艇/精神/形成/及其/本质特征/进行/剖析/\n/第九章/关于/黄旭华/和谐/家庭/幸福/生活/反映/家人/事业/坚定/支持/理解/本章/他/夫人/李世英/进行/重点/介绍/突出/反映/她/品德/品质/品位/李世英/塑造/一个/近乎/完美/黄旭华/同时/也/丰富/自己/人生/提升/自己/人生价值/诠释/作为/知识/女性/另外/一种/成功/

```
[23]: # 重新生成词云
from wordcloud import WordCloud
wordcloud_cn = WordCloud(font_path="simsun.ttf").generate(final)
%pylab inline
import matplotlib.pyplot as plt
plt.imshow(wordcloud_cn, interpolation='bilinear')
plt.axis('off')
```

PyLab is deprecated, use %matplotlib inline and import the required libraries.
Populating the interactive namespace from numpy and matplotlib

```
[23]: (-0.5, 399.5, 199.5, -0.5)
```



好了！是不是感觉自己很不错呢

我们还是换一个形状，换一个黄院士的头像吧，我已经替你下载好啦，在文件夹里面

给词云加上特定的形状

```
[ ]: big_pic = imread('huangxuhua.jpg')

[ ]: wordcloud_cn_x = WordCloud(mask=big_pic,width=500,height=300, background_color='white',scale=2.5, font_path="simsun.ttf").generate(final)
plt.imshow(wordcloud_cn_x)
plt.axis('off')
```

可以在文件夹中打开，是一个很大的图，也很清楚

任务四：运行词云图代码

任务四：运行词云图代码



注意：以下有wordcloud所有参数的设置，供大家作业的时候参考，修改词云效果。



第四讲 情感分析

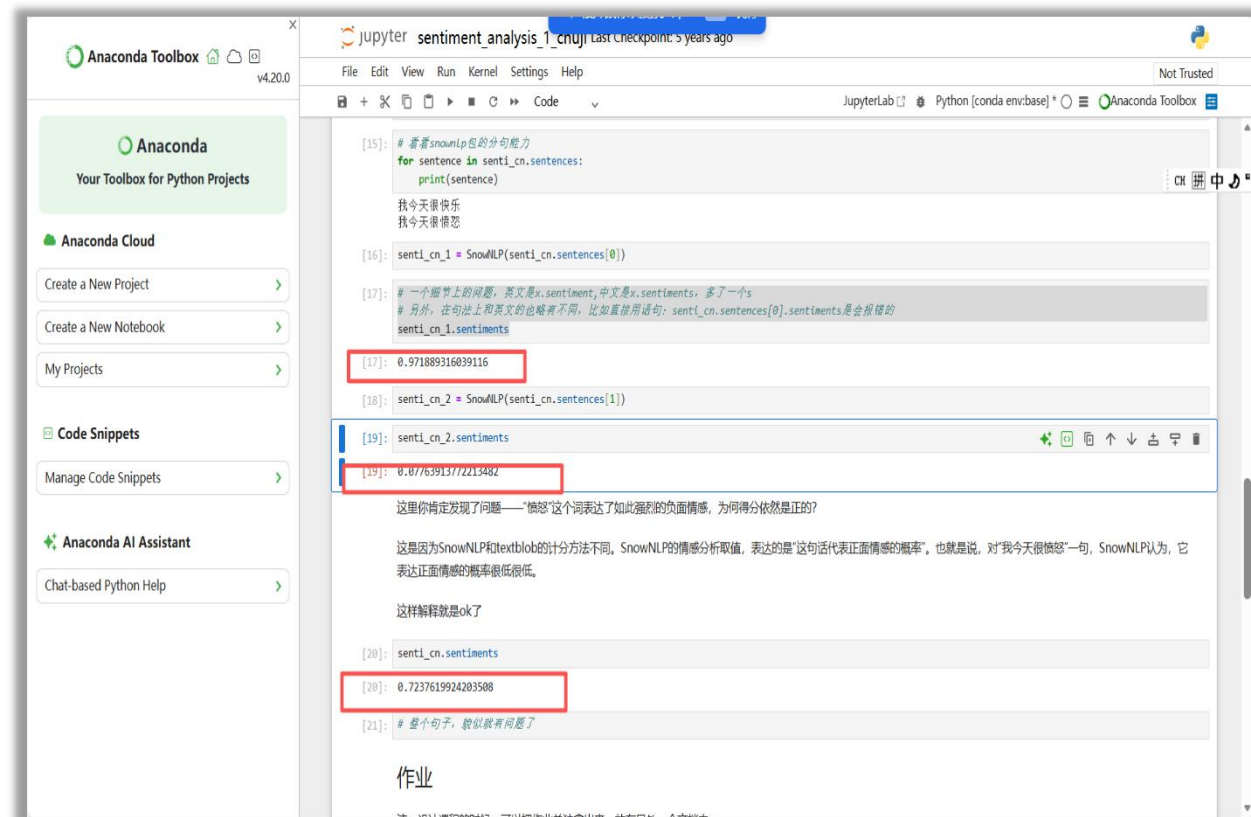
任务一：情感分析



第四讲 情感分析

任务二：四段代码的运行

第一段代码



```
[15]: # 查看snownlp包的分割句能力
for sentence in senti_cn.sentences:
    print(sentence)

我今天很快乐
我今天很愤怒

[16]: senti_cn_1 = SnowNLP(senti_cn.sentences[0])

[17]: # 一个细节上的问题，英文是x.sentiment，中文是x.sentiments，多了一个s
# 另外，在句法上和英文的也略有不同，比如直接用语句：senti_cn.sentences[0].sentiments是会报错的
senti_cn_1.sentiments

[17]: 0.971889316039116

[18]: senti_cn_2 = SnowNLP(senti_cn.sentences[1])

[19]: senti_cn_2.sentiments

[19]: 0.07763913772213482

这里你肯定发现了问题——“愤怒”这个词表达了如此强烈的负面情绪，为何得分依然是正的？

这是因为SnowNLP和textblob的计分方法不同，SnowNLP的情感分析取值，表达的是“这句话代表正面情感的概率”。也就是说，对“我今天很愤怒”一句，SnowNLP认为，它表达正面情感的概率很低很低。

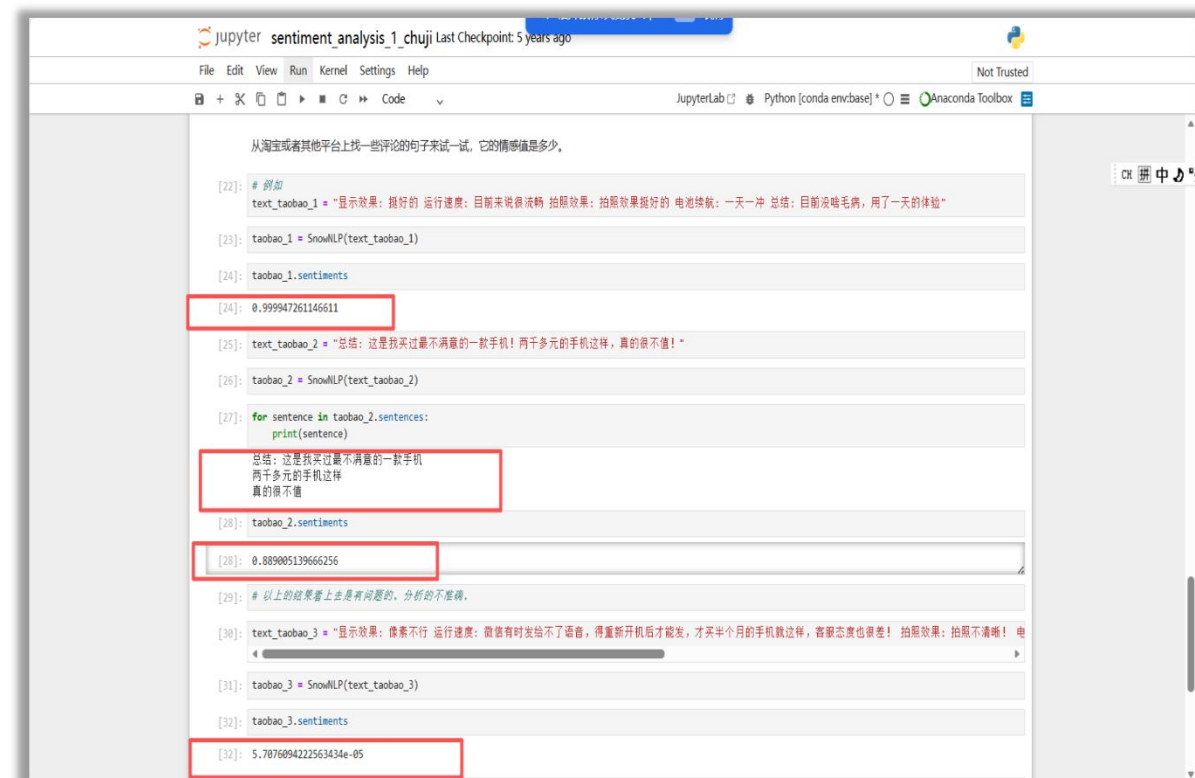
这样解释就是ok了

[20]: senti_cn.sentiments

[20]: 0.7237619924203508

[21]: # 整个句子，貌似就有问题了

作业
```



```
从淘宝或者其他平台上找一些评论的句子来试一试，它的情感值是多少。

[22]: # 例如
text_taobao_1 = "显示效果：挺好的 运行速度：目前来说很流畅 拍照效果：拍照效果挺好的 电池续航：一天一冲 总结：目前没啥毛病，用了一天的体验"

[23]: taobao_1 = SnowNLP(text_taobao_1)

[24]: taobao_1.sentiments

[24]: 0.999947261146611

[25]: text_taobao_2 = "总结：这是我买过最不满意的一款手机！两千多元的手机这样，真的很不值！"

[26]: taobao_2 = SnowNLP(text_taobao_2)

[27]: for sentence in taobao_2.sentences:
    print(sentence)

总结：这是我买过最不满意的一款手机
两千多元的手机这样
真的很不值

[28]: taobao_2.sentiments

[28]: 0.889005139666256

[29]: # 以上的结果看上去是有问题的，分析的不准确。

[30]: text_taobao_3 = "显示效果：像素不行 运行速度：微信有时发不了语音，得重新开机后才能发，才买半个月的手机就这样，客服态度也很差！ 拍照效果：拍照不清晰！电"

[31]: taobao_3 = SnowNLP(text_taobao_3)

[32]: taobao_3.sentiments

[32]: 5.7876094222563434e-05
```

第四讲 情感分析

任务二：四段代码的运行

第二段代码

```
Jupyter sentiment_analysis_2_timeline Last Checkpoint: 2 years ago
File Edit View Run Kernel Settings Help
JupyterLab Python [conda env:base] Anaconda Toolbox

反之，如果你获取到的时间只精确到日期，例如“2017-04-20”这样，那么Pandas只会把它当成字符串，后面的时间序列分析无法使用字符串数据。解决办法是在这里加入以下两行代码：

from dateutil import parser
df["date"] = df.date.apply(parser.parse)

这样，你就获得了正确的时间数据了

[6]: text = df.comments.iloc[0] # iloc是索引（用来定位到指定的位置），这里就是第一条评论文本
[7]: text
[7]: '这家最爱吃的火锅，一星期必吃一次啊！最近才知道他家还有免费鸡架.....炒鸡好吃炒鸡嫩啊！！新出的红皮土豆也好好吃，还有炸酥肉，秒杀任何火锅店啊！服务员太可爱，告诉我们半份豆花是4块儿，一份豆花是6块儿，点两个半份比较合适，太实在了哈哈，每次妈妈开开心心果吃服务员都给我们打包带走>>希望A餐厅早日出爆款，期待ing...'
[8]: from snowlp import SnowNLP
[9]: s = SnowNLP(text)
[11]: s.sentiments
[11]: 0.4244401830222834

情感分析数值可以正确计算。在此基础上，我们需要定义函数，以便批量处理所有的评论信息。

[12]: def get_sentiment_cn(text):
    s = SnowNLP(text)
    return s.sentiments # snowlp是通用的，就设那么准确

然后，我们利用Python里面强大的apply函数，来一次性处理所有评论，并且将生成的情感数值在数据框里面单独存为一列，称为sentiment。

[13]: df["sentiments"] = df.comments.apply(get_sentiment_cn) #这行代码很有意思！
[14]: df.head(100)
```



第四讲 情感分析

任务二：四段代码的运行

第三段代码

```
jupyter sentiment_analysis_3_大模型_健康文本细粒度情感抽取 Last Checkpoint: 7 months ago
File Edit View Run Kernel Settings Help
JupyterLab Python [conda env:base] Anaconda Toolbox

以下是一段采用大语言模型处理“心理-身体”病痛文本，细粒度情感/感受/症状实体与实体关系提取的工作

[1]: import requests
import json

# DeepSeek API 端点
url = "https://api.deepseek.com/v1/chat/completions"

# 替换为您的 DeepSeek API 密钥
API_KEY = "sk-Bed0b579a89a40f88e22cf7bc12eeca7" # 直接复制过来

# 请求头，包含 API 密钥和内容类型
headers = {
    "Authorization": f"Bearer {API_KEY}",
    "Content-Type": "application/json"
}

[2]: # 患者描述文本
text = {
    "我今年58岁，退休后感到生活失去了重心，开始出现失眠、头痛和疲乏无力的症状。"
    "后来，皮肤变得异常敏感，连衣服的触碰都像针扎一样疼痛。"
    "我常常感到心慌、胸闷，背部沉重得像压了一块石头。"
    "对光线和声音变得极度敏感，电话铃声都会让我惊恐。"
    "多次到医院检查，结果显示没有器质性疾病。"
    "我变得不愿出门，不想与人交流，整天把自己关在屋里，拉紧窗帘，感觉生活毫无意义。"
}

# 构建提示词，要求模型提取细粒度情感实体
prompt = (
    "请从以下患者描述中提取出具体的身体部位、症状以及对应的情感状态，"
    "并以 JSON 格式返回，格式如下："
    "{'实体': [{'部位': '...', '症状': '...', '情感': '...', ...}]\n\n"
    f"患者描述: {text}"
)
```

```
jupyter sentiment_analysis_3_大模型_健康文本细粒度情感抽取 Last Checkpoint: 7 months ago
File Edit View Run Kernel Settings Help
JupyterLab Python [conda env:base] Anaconda Toolbox

[3]: # 请求体，包含模型参数和提示词
data = {
    "model": "deepseek-chat",
    "messages": [
        {"role": "user", "content": prompt}
    ],
    "temperature": 0.3
}

try:
    # 发送 POST 请求
    response = requests.post(url, headers=headers, data=json.dumps(data))

    # 检查响应状态码
    if response.status_code == 200:
        # 解析 JSON 响应
        result = response.json()
        # 提取模型生成的内容
        generated_text = result['choices'][0]['message']['content']
        print("细粒度情感实体抽取结果:")
        print(generated_text)
    else:
        # 处理错误响应
        print(f"请求失败，状态码: {response.status_code}")
        print(f"错误信息: {response.text}")

except requests.exceptions.RequestException as e:
    # 处理网络请求异常
    print(f"网络请求失败: {e}")
except json.JSONDecodeError as e:
    # 处理 JSON 解析异常
    print(f"JSON 解析失败: {e}")
except Exception as e:
    # 处理其他异常
    print(f"发生未知错误: {e}")

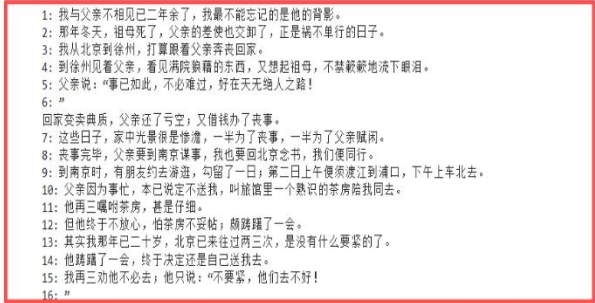
# 细粒度情感实体抽取结果:
{
  "实体": [
    {
      "部位": "全身",
      "症状": "失眠",
      "情感": "失去重心"
    },
    {
      "部位": "头部",
      "症状": "头痛",
      "情感": "疲乏无力"
    },
    {
      "部位": "皮肤",
      "症状": "皮肤敏感，触碰如针扎疼痛",
      "情感": "不适"
    },
    {
      "部位": "心脏/胸部",
      "症状": "心慌、胸闷",
      "情感": "惊恐"
    },
    {
      "部位": "背部",
      "症状": "沉重如压石头",
      "情感": "不适"
    },
    {
      "部位": "感官系统",
      "症状": "对光线和声音极度敏感",
      "情感": "惊恐"
    },
    {
      "部位": "全身",
      "症状": "不愿出门，不想与人交流，拉紧窗帘",
      "情感": "生活毫无意义"
    }
  ]
}

提问:
1. 请问你觉得大语言模型在识别患者的身体、心理、情感的工作中，表现如何?
2. 除了健康领域的情感世界理解与识别，你还能够想到哪些其他重要的领域，可以开展类似的工作呢?
```

第四讲 情感分析

任务二：四段代码的运行

第四段代码



```
[1]: import requests
import time
import re

API_KEY = "sk-8ed0b579a89a40f88e22cf7bc12eeca7" #替换成你自己的密钥
API_URL = "https://api.deepseek.com/v1/chat/completions"

headers = {
    "Content-Type": "application/json",
    "Authorization": f"Bearer {API_KEY}"
}

emotion_results = []

for num, sent in numbered_sentences:
    prompt = f"""你是一个情感分析助手。请对下面这句话做如下分析：

1. 提取或生成一个最能代表情绪的词语（只能一个）；
2. 对整句话的情绪进行打分，范围是 -1（非常负面）到 1（非常正面），中性为 0。

直接用如下格式输出：
情感词: XXX
情感值: X.X

句子: {sent}"""

    data = {
        "model": "deepseek-chat",
        "messages": [{"role": "user", "content": prompt}],
        "temperature": 0.3
    }
```


第四讲 情感分析

任务二：四段代码的运行

四段代码总结

通过对四段情感分析代码的运行与整合，形成了一套从基础处理到深度挖掘的完整文本情感分析 workflow。第一阶段的SnowNLP验证了其快速分句与基础情感极性判断能力，适用于海量文本的初步筛选。第二阶段在此基础上，实现了对Excel表格的批量自动化处理与情感趋势可视化，能高效呈现文本集的整体情感波动。

为进一步提升分析深度，第三阶段引入大语言模型，专注于对“心理-身体”类复杂文本进行细粒度实体与关系抽取，能够识别隐含的情感与症状并构建其关联网络，超越了单纯的情感极性分类。最终，第四阶段综合运用了分句、大模型实体提取与情感计算技术，并创新性地绘制了情感轨迹图，以时间序列形式直观展现了文学作品中情感词及其强度的演变过程。

整体上，这项工作流体现了“广度”与“深度”的结合：SnowNLP保障了处理效率，大语言模型则提供了深层的语义理解与知识挖掘能力。可视化方案将数据结果转化为直观洞察。未来可通过两级分析策略（先快速筛选，再对关键部分深度分析）进一步优化性能与成本的平衡，该流程适用于学术研究、舆情洞察及临床文本分析等多个专业领域。



南京工业大学
NANJING TECH
UNIVERSITY

02 关联数据分析

吴志祥

18205185639

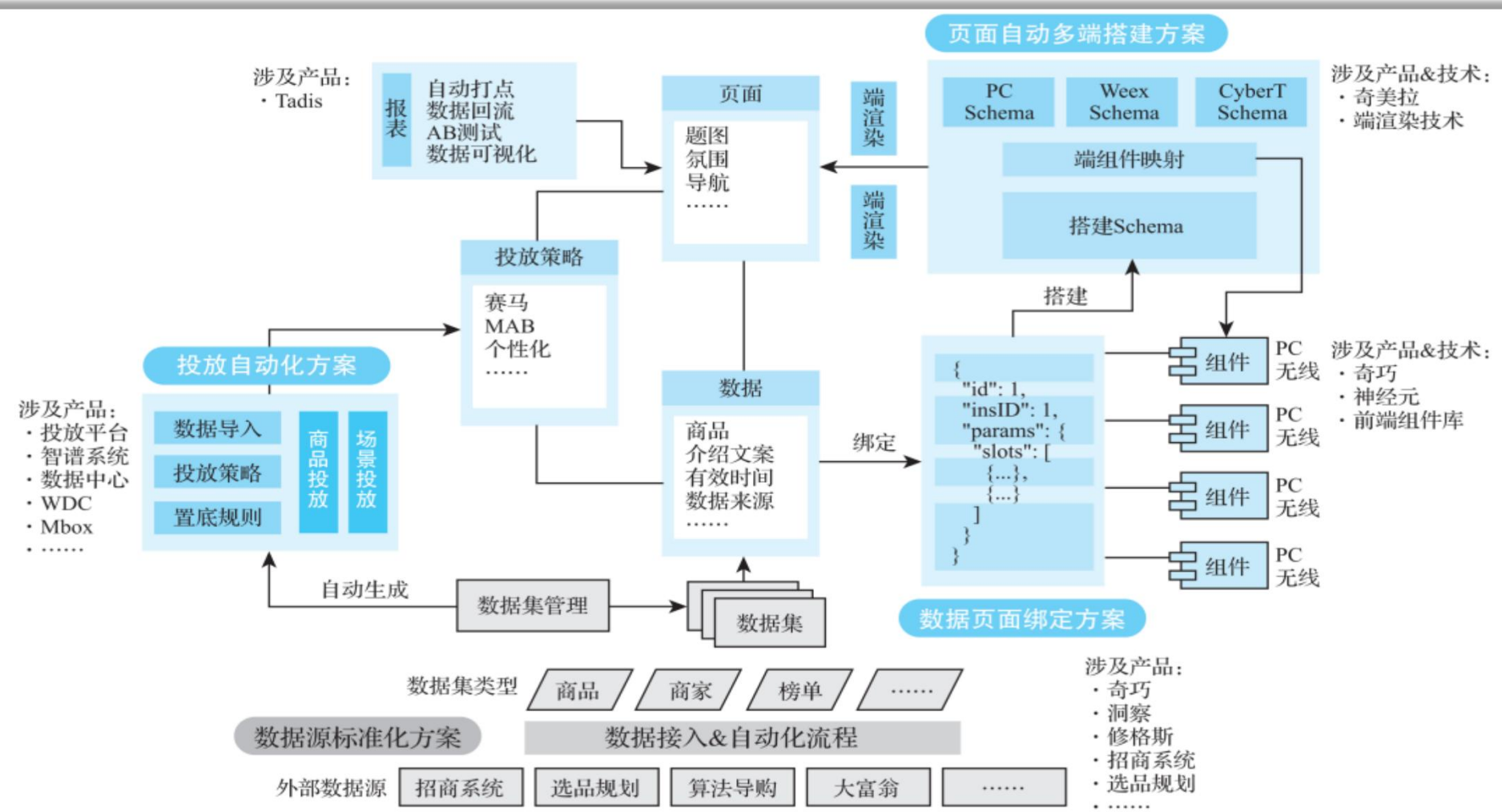
cnwzx2012@njtech.edu.cn



第六讲 知识图谱理念

任务一：阿里商品大脑

阿里商品大脑知识图谱的构建：

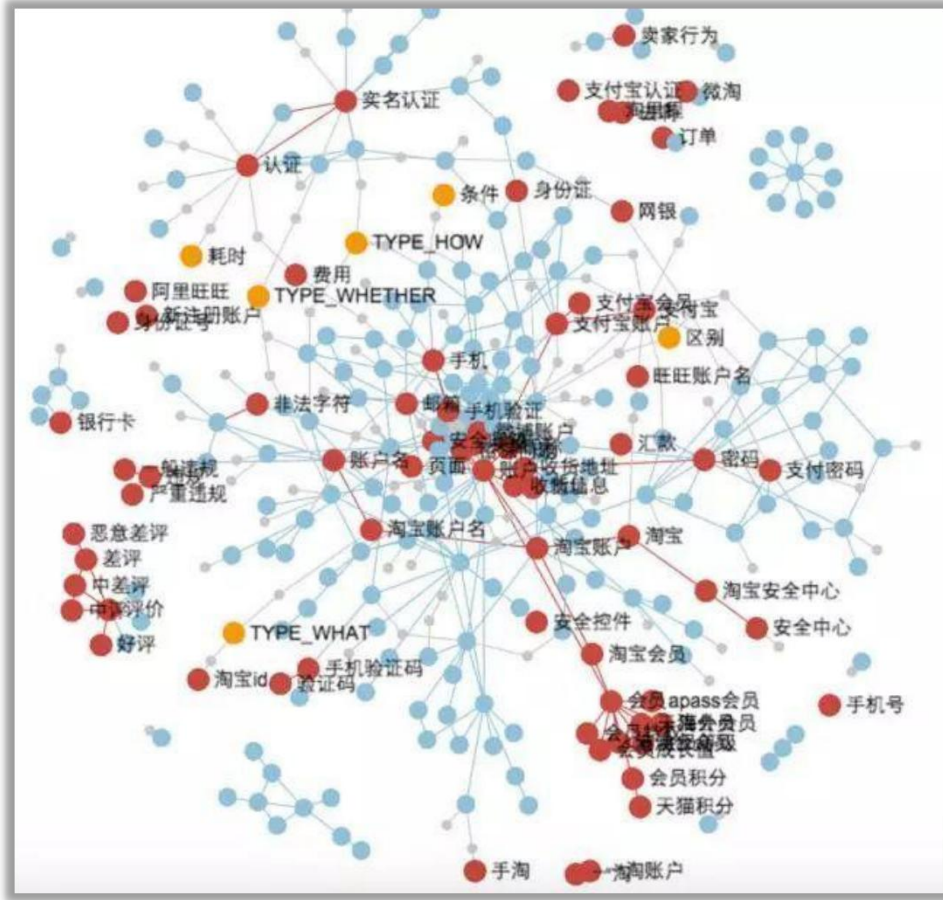


知识图谱的构建

第六讲 知识图谱理念

任务一：阿里商品大脑

阿里商品大脑知识图谱的简介：



知识图谱简介：阿里商品大脑知识图谱是阿里巴巴集团构建的超大规模、多模态商业认知智能系统。它本质上是电商领域的“结构化商业常识库”，旨在将散乱、非结构化的商品信息转化为机器可理解、可推理的关联知识网络，从而驱动阿里生态内各项业务的智能化。

核心定位与规模：该知识图谱是阿里电商业务的底层智能基础设施。截至最近的数据，它已覆盖数十亿级的商品实体，关联了超过千亿级的关系三元组，融合了商品、品牌、品类、卖点、成分、适用人群乃至多媒体信息，是国内规模最大、应用最复杂的领域知识图谱之一。

关键特征与能力

多模态融合：不仅是文本属性的关联，更整合了商品主图、详情图、直播视频片段等视觉信息，形成“图文音视”一体化的知识表达，以支持直播、短视频等新场景。

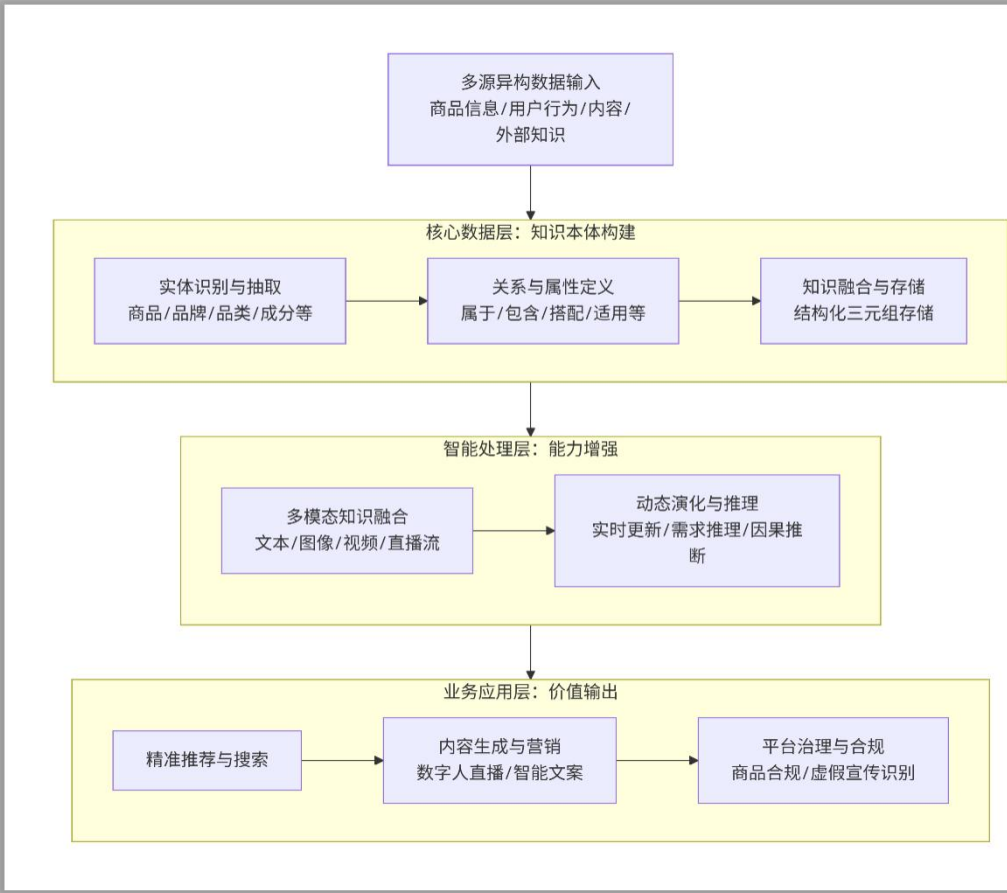
深度认知推理：其目标超越基础信息关联，致力于实现深度的用户需求理解与商业逻辑推理。例如，能通过分析“冬季、干性皮肤、抗衰”等多个标签，推理并推荐成分匹配的精华产品。

实时演化与开放生态：知识图谱持续通过交易、用户行为、内容数据进行动态更新。同时，阿里通过“藏经阁”计划、开放部分数据集（如AliOpenKG）等方式，与学术界、产业界共建生态，推动行业标准。为阿里内部大模型（如通义千问）提供精准、可信的商业知识注入，解决大模型在电商领域的幻觉问题。

第六讲 知识图谱理念

任务一：阿里商品大脑

阿里商品大脑知识图谱的分析：



（一）核心数据层：知识本体构建

这是图谱的“骨骼”与“血肉”，负责将原始数据转化为结构化知识。

实体体系：定义了电商领域最核心的实体类型，包括：

商品：核心实体，粒度可细至SKU。**品牌与店铺：**商品的归属与信用实体。

品类与属性：构成商品分类体系的“垂直”维度（如“手机>屏幕尺寸>6.5英寸”）。

成分、功效、人群：尤其在美妆、食品、保健品等领域形成的“横向”知识维度（如“玻尿酸→补水→干性皮肤”）。

内容与媒体：将商品描述、用户评价、直播视频、详情图也作为可关联的实体。

关系与属性网络：实体之间通过丰富的关系连接形成网络。

典型关系包括：

隶属关系：如 商品A -[属于]-> 品类B。

属性关系：如 商品A -[拥有成分]-> 玻尿酸。

搭配/替代关系：如 口红C -[搭配]-> 妆容D，基于消费数据挖掘。

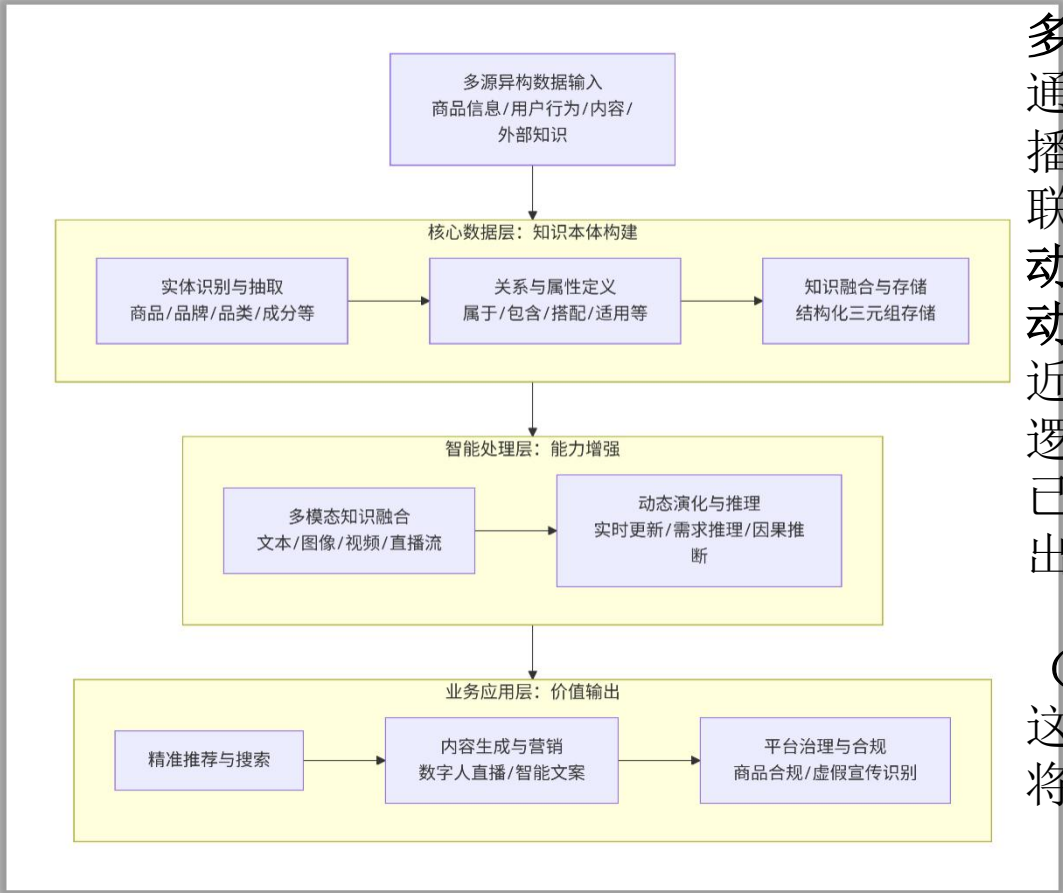
共现/因果关系：如 差评关键词E -[导致]-> 退货率上升，用于深度分析。

知识的表示与存储：以上述实体和关系构成“三元组”（头实体-关系-尾实体），并存储于图数据库等系统中，支持高效的关联查询和路径推理。

第六讲 知识图谱理念

任务一：阿里商品大脑

阿里商品大脑知识图谱的分析：



（二）智能处理层：能力增强

这是让知识“活”起来并持续生长的“神经系统”。

多模态知识融合引擎：这是其前沿性的关键。系统不仅处理文本，还通过CV技术理解商品主图的场景、风格，通过视频分析技术拆解直播中的口播卖点和演示片段，并将这些视觉、听觉特征与商品实体关联，形成跨模态的统一知识表达。

动态演化与推理引擎：图谱不是静态的。

动态更新：新的商品、用户搜索行为、交易趋势、热点内容会实时或近实时地反馈到图谱中，调整实体权重或发现新关系。

逻辑推理：基于已构建的关系网络，可以进行隐式知识推理。例如，已知“商品A适用于油性皮肤”且“用户B是油性皮肤”，则可推理出“A可能适合B”，即使两者从未直接关联。

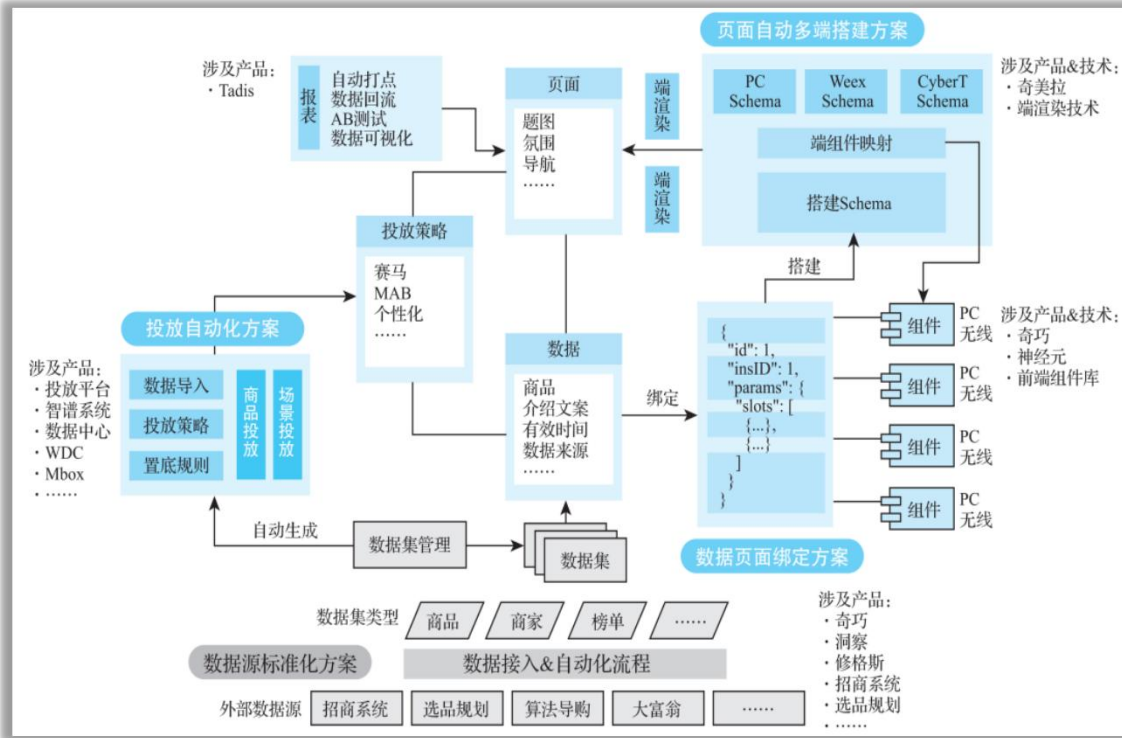
（三）业务应用层：价值输出

这是知识产生价值的“决策与执行器官”。它通过API或嵌入式模块，将知识能力赋能给各类业务场景。

第六讲 知识图谱理念

任务一：阿里商品大脑

阿里商品大脑知识图谱的核心特征：



规模与复杂性：它处理的是超大规模、高复杂度的电商领域数据。数十亿实体和千亿级关系，远超通用知识图谱。其复杂性不仅体现在数量上，更在于电商领域特有的、精细且动态变化的类目体系和属性关系网络。

多模态与融合性：这是其最显著的技术特征。它打破了传统知识图谱以文本为主的局限，实现了文本、图像、视频、直播流、交易数据等多源信号的统一对齐与语义融合。这使得知识不仅能回答“这是什么”，还能回答“它看起来怎么样”、“如何使用”等更丰富的问题。

动态演化与智能性：它是一个“活”的、自演化的生态系统。知识并非一次性构建完成，而是随着平台运营、市场变化和用户行为持续迭代。其智能性体现在不仅能被动查询，更能主动进行需求推理、趋势预测和因果分析，从“记录知识”走向“创造知识洞察”。

强业务导向与场景驱动：与学术研究中的通用图谱不同，它的构建和发展始终以解决实际商业问题为第一目标。无论是为了提升搜索转化、优化推荐效果、赋能内容创作还是加强平台治理，每一个技术迭代和知识维度的扩展，都有明确的业务场景和价值衡量标准。

生态开放与基建化：阿里巴巴正有意识地将该知识图谱部分能力“基础设施化”。通过开放数据集（AliOpenKG）、举办算法大赛、开展“藏经阁”等产学研合作，它试图在输出行业标准的同时，吸引外部智力反哺自身，构建更繁荣的电商认知智能生态。

第六讲（2）知识图谱工具

任务一：知识图谱链接平台，检索

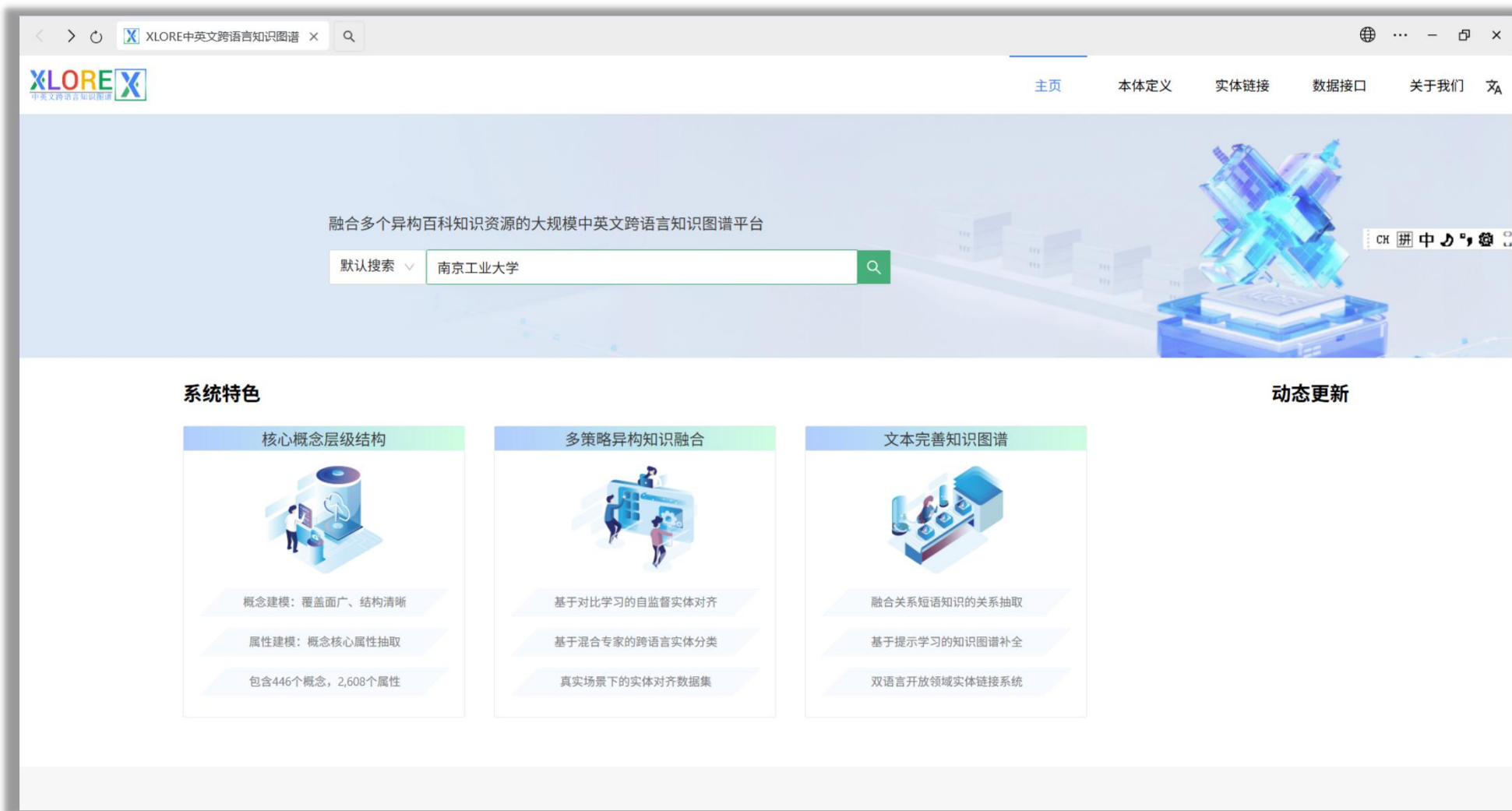
OpenKG



第六讲 (2) 知识图谱工具

任务一：知识图谱链接平台，检索

清华大学-XLORE



第六讲 (2) 知识图谱工具

任务一：知识图谱链接平台，检索

知识图谱-思知

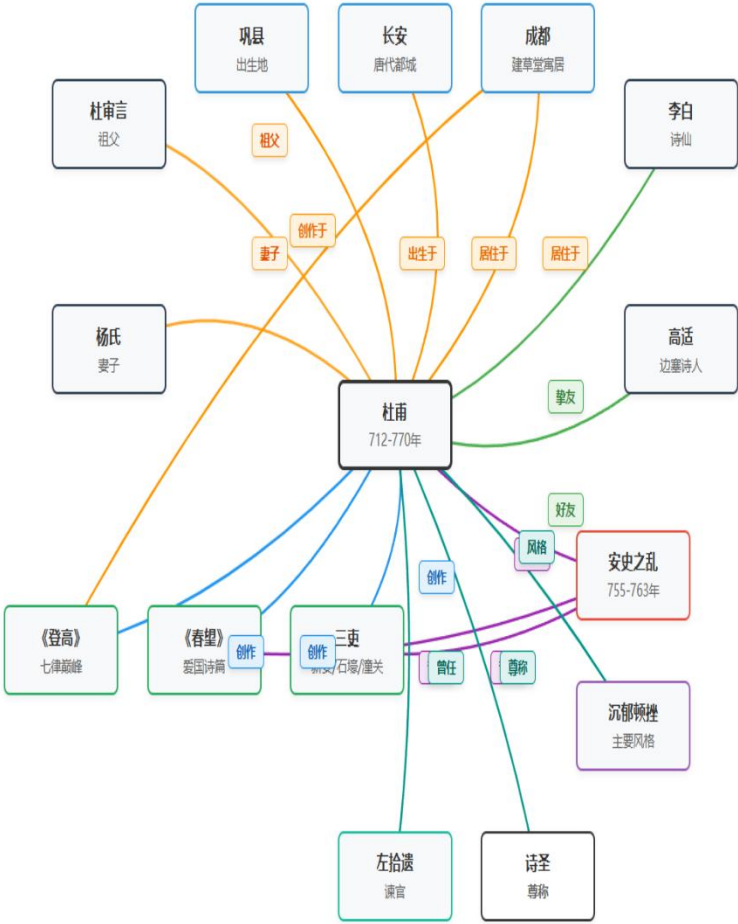


第六讲（2）知识图谱工具

任务二：白板建模

使用draw.io进行白板建模

本杜甫知识图谱白板建模是一个交互式知识可视化界面，以唐代诗人杜甫为核心节点，系统展示其生平、创作与历史关联的多维关系网络。图谱围绕杜甫这一中心人物，整合了家族关系（祖父杜审言、妻子杨氏）、重要社会交往（诗友李白、高适）、代表作品（《登高》《春望》《三吏》三组史诗）、地理轨迹（出生地巩县、寓居地长安与成都）、历史背景（安史之乱）、艺术风格（沉郁顿挫）、历史称号（诗圣）及官职经历（左拾遗）等八类实体节点，并通过六种颜色编码的关系线（家族、朋友、创作、地理、历史、特征关系）清晰呈现实体间的语义连接。该建模采用贝塞尔曲线优化布局，支持按关系类型筛选查看、节点详情交互查询及数据导出功能，旨在为用户提供一个结构清晰、操作直观的知识探索工具，以可视化方式揭示杜甫作为“诗圣”在文学、历史与社会网络中的多维定位。

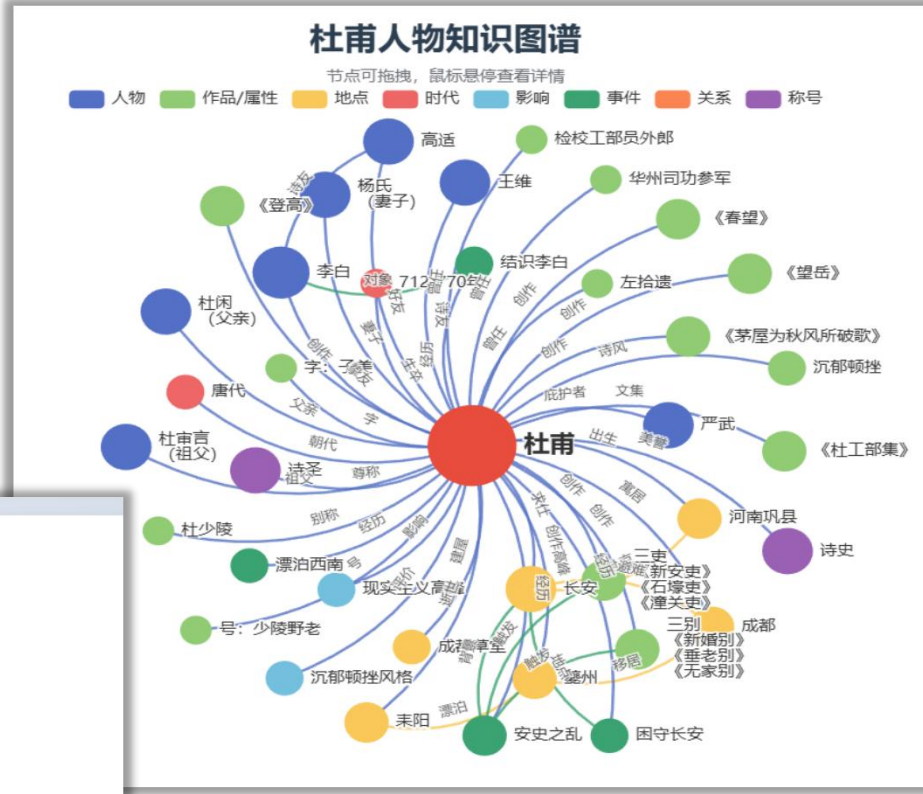
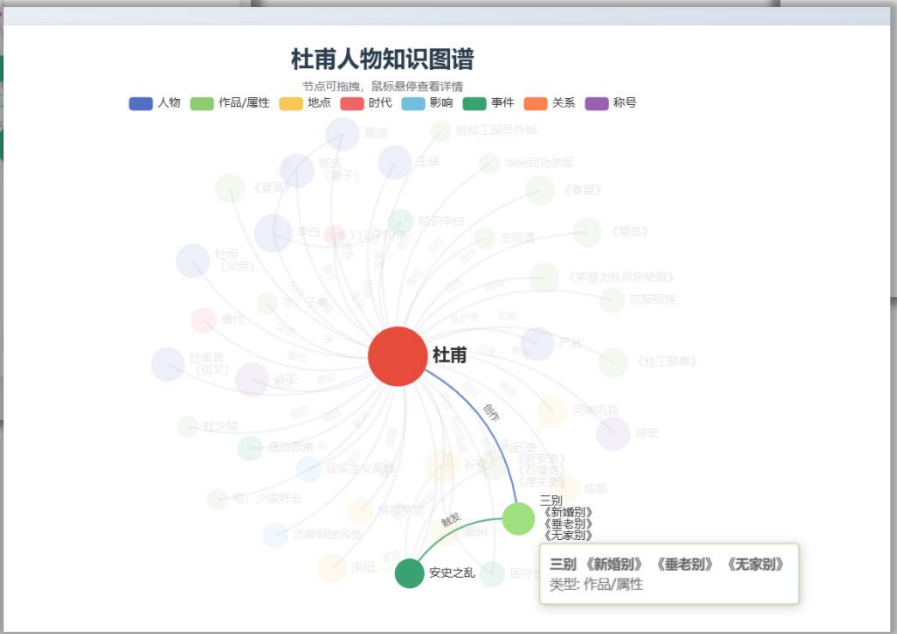
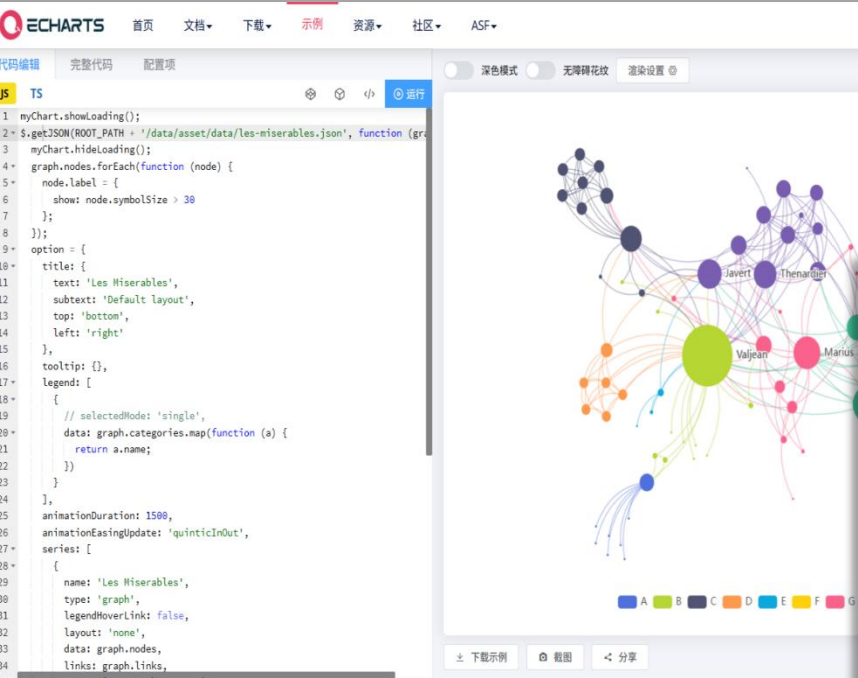


第六讲 (2) 知识图谱工具

任务三：使用Echarts关系图制作知识图谱

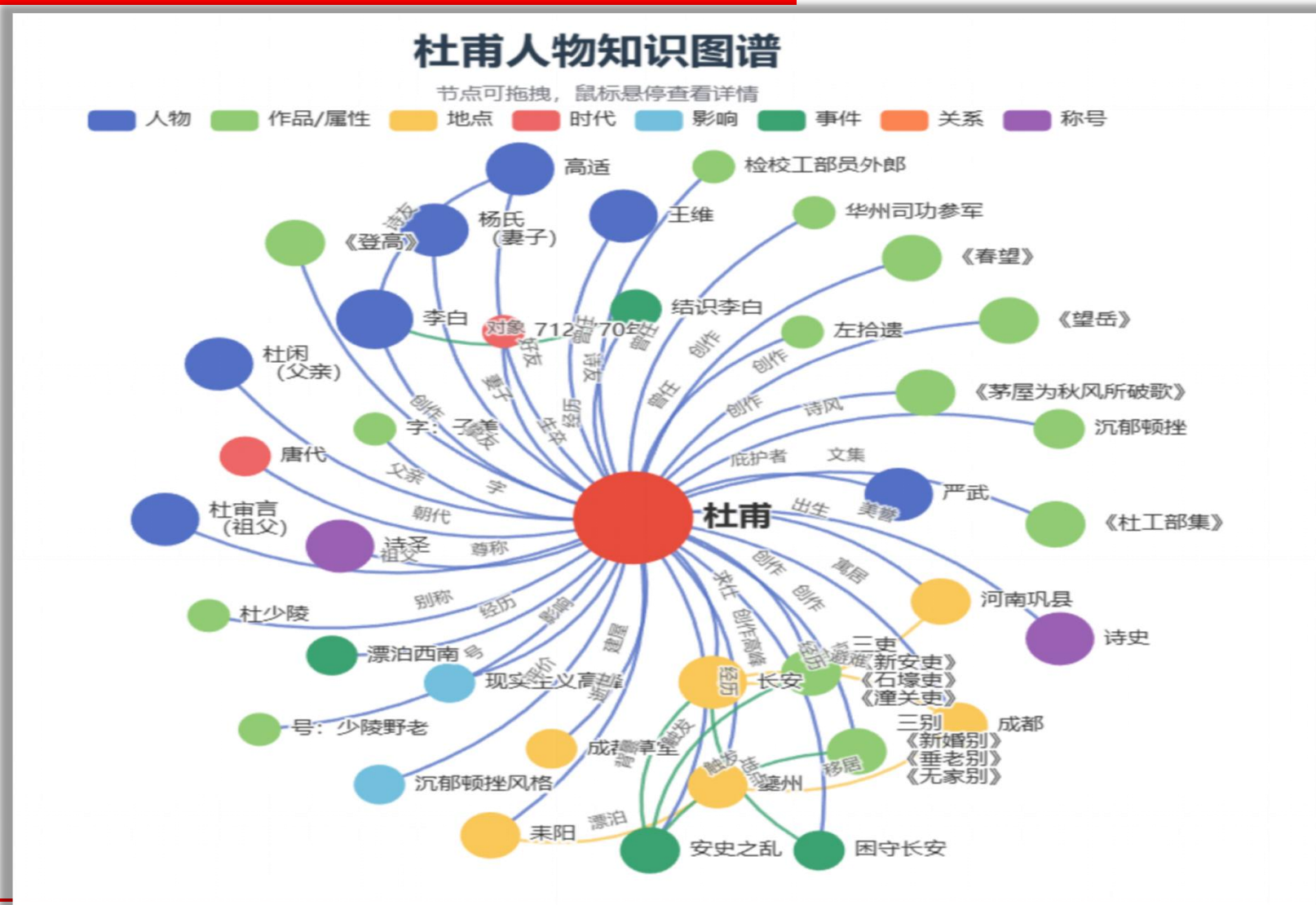
根据Echarts关系图代码修改

得到杜甫的人物知识图谱



第六讲（2） 知识图谱工具

任务三：使用Echarts关系图制作知识图谱



第六讲（2）知识图谱工具

任务四：使用Neo4j编程绘制一款（简单）知识图谱

钱钟书的人物关系知识图谱

